IE University

School of Science & Technology

The Impact of Post-Training Techniques on Feature Representations

Alberto Puliga

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Prof. E. Castelló Ferrer

Connection Science, MIT, Cambridge, USA

Robotics & AI Lab, School of Science & Technology, IE University, Spain

Abstract

Large language models reach users not as raw next-token predictors but through post-training: instruction tuning, reward modelling, and preference optimisation layered on top of a base model trained on web text. Most mechanistic interpretability work, however, has studied base models, leaving the internal effects of post-training largely unexamined at the representational level. This thesis asks whether instruction tuning introduces new features into a language model's internal representations or mainly reorganises existing ones.

The question is tested using matched Sparse Autoencoders from the Gemma Scope 2 release, which provides directly comparable feature dictionaries for the pre-trained and instruction-tuned variants of Gemma 3 1B. A four-stage pipeline collects residual-stream activations from both checkpoints over 5,000 FineWeb sequences, classifies features into preserved, modified, and variant-exclusive categories using decoder cosine similarity and activation correlation, attaches descriptive labels to a frequency-weighted sample through dual-LLM consensus annotation with inter-annotator reliability analysis, and cross-references the classification against differential activation tests on three behavioural probe datasets covering safety, instruction-following, and truthfulness.

Cleanly preserved features turn out to be a small minority of the active feature set. The features most relevant to instruction-shaped inputs and harmful content are predominantly already present and selective in the base model; instruction tuning amplifies and reweights them rather than introducing them. These results provide the first feature-level evidence from a real instruction-tuned language model consistent with the "wrapper hypothesis" of Jain et al. (2024), previously tested only in synthetic settings, and suggest that instruction tuning reorganises a pre-existing representational substrate rather than installing new computational structure. This framing has implications for how alignment teams reason about the base-model origins of deployed safety behaviours.

1 Introduction

1.1 Problem and Motivation

Contemporary large language models reach users not through pre-training alone but through a post-training pipeline that layers supervised instruction tuning, preference learning from human feedback, constitutional AI, and direct preference optimisation on top of a base model trained to predict next tokens on web-scale text [2, 6, 23, 29, 39]. Post-training, not the base model, is what gives the deployed system its characteristic behaviours: following instructions, answering in a particular register, refusing harmful requests, staying in the role of an assistant. Yet most mechanistic interpretability research has concentrated on base models, from Olah et al. [22]'s Zoom In essay on circuits to the indirect object identification circuit of Wang et al. [34], the factual association edits of Meng et al. [19], and the feed-forward key–value memories of Geva et al. [11]. The internal mechanics of post-training remain comparatively under-studied. There is, in other words, a gap between the layer of the language model that interpretability understands best and the layer that users actually interact with.

This gap matters beyond scientific completeness. If we do not understand at a representational level what instruction tuning does to a model, we cannot tell whether an observed safety behaviour is a robust property of the model's computation or a brittle veneer, removable by further fine-tuning, jailbroken by adversarial prompting, or silently circumvented by deceptive alignment [12, 40]. Right now the answer to these questions is largely behavioural: probe the deployed model, observe whether it refuses harmful prompts, draw conclusions. Informative, but an outside-the-box view. What we lack is an inside-the-box account of which internal representations the post-training stack builds, reuses, or rearranges, and which of them actually carry the behaviours we care about.

The mechanism of instruction tuning is the subject of an empirically answerable debate. One widely cited position is the superficial alignment hypothesis of Zhou et al. [38]: a surprisingly small amount of high-quality instruction-following data suffices to produce strong instruction-following behaviour, implying that instruction tuning may not add new represen-

tational content so much as teach the model to expose existing content in a task-appropriate register. A competing position, less often named but implicit in much of the alignment literature, holds that instruction tuning installs new computational structure (new features, circuits, or decision procedures) that did not previously exist in the base model and that are necessary for instruction-tuned behaviour. These two positions predict different things about the model's internal representations, and whether a specific feature in the instruction-tuned model's layer 13 residual stream has a counterpart in the base model's layer 13 residual stream is exactly the kind of observation that can distinguish them.

Until recently, testing this distinction at the representational level was impractical. The tools of mechanistic interpretability (probing classifiers, activation patching, attention pattern inspection, residual stream decomposition) either measured aggregate behavioural properties or studied individual circuits one at a time; neither gave a vocabulary for systematically comparing two models' entire representational states. Sparse Autoencoders changed this. Beginning with the toy-model superposition work of Elhage et al. [9] and early monosemanticity results [7], and maturing through the Gemma Scope release [17], SAEs now provide a feature-level dictionary of a model's residual-stream activations, where each entry corresponds to a direction in activation space that tends to activate on semantically legible patterns. Two checkpoints of the same model, before and after post-training, can be equipped with SAEs trained on the same residual-stream location and their feature dictionaries compared directly. A feature that exists in both dictionaries, points in the same direction, and activates on the same inputs has been preserved across post-training. A feature that exists in only one dictionary, or whose direction or functional behaviour has shifted, has been modified, added, or removed. This comparison operationalises the debate between the superficial-alignment reading and the new-capability reading of instruction tuning, and it is what this thesis undertakes.

Why study instruction tuning specifically, rather than the full post-training stack? Practically, instruction tuning is the first and simplest stage of post-training, and its effects can be isolated from reinforcement-learning-based stages provided that matched Sparse Autoencoders exist for both the base model and the supervised-fine-tuned checkpoint. The Gemma Scope 2 release is, at the time of writing, the only matched-SAE release for any production-class lan-

guage model, and the Gemma 3 1B base and instruction-tuned checkpoints are its only covered pair, a constraint that determines the model selection of this thesis (see Subsection 2.1). On the scientific side, instruction tuning is also the stage whose mechanism is most directly addressed by the superficial-alignment literature and the recent mechanistic fine-tuning work [8, 13, 27], so a feature-level study connects to an existing debate rather than starting one from scratch.

The research question, stated in its final form in Subsection 1.2, can be previewed here: at the feature level, does instruction tuning introduce new features into a language model's internal representations, or does it reorganise existing ones? The rest of this thesis answers that question for one model pair, one interpretability tool, one layer, three behaviourally grounded probe datasets, with explicit attention to robustness across reasonable perturbations of the free parameters. In anticipation of Section 4: the answer is consistent with the reorganisation reading. The behaviourally relevant features of the instruction-tuned model are predominantly reshaped versions of features already present in the base model, not new features introduced from scratch. The conclusion speaks to one layer of one model and is correlational rather than causal, but it is a concrete, reproducible data point on a question that has so far been addressed mostly at the behavioural and circuit level, and it provides a methodology reusable as matched Sparse Autoencoders become available for other model pairs.

1.2 Prior Art and Research Question

This subsection reviews the prior work that frames the research question. It is organised thematically in four parts. Subsubsection 1.2.1 covers representational foundations: the linear representation hypothesis, superposition, and dictionary learning as responses to the problem that individual neurons are not the right unit of analysis. Subsubsection 1.2.2 traces Sparse Autoencoders from early toy models to the Gemma Scope releases that make this work possible. Subsubsection 1.2.3 surveys what interpretability currently knows about post-training's effect on internal representations, covering the superficial-alignment argument, mechanistic fine-tuning work, and representation engineering. Subsubsection 1.2.4 identifies the gap this thesis addresses and states the research question.

1.2.1 Representational Foundations: Superposition, Linear Features, and the Case for Dictionary Learning

The unit-of-analysis problem in mechanistic interpretability predates the field itself: it was already implicit in work on distributed word representations [20], which showed that directions in an embedding space carry semantically meaningful structure not readable from any individual coordinate. Individual neurons are generally not the right unit of analysis because the same neuron participates in many unrelated computations. Radford, Jozefowicz, and Sutskever [28] observed this in character-level language models, where a single "sentiment neuron" was identified but turned out to be the exception rather than the rule. Olah, Mordvintsev, and Schubert [21] and Olah et al. [22] articulated the broader view that the right unit of analysis is the feature, a direction in activation space corresponding to a semantically legible pattern, and that features need not correspond to individual neurons.

The formal explanation for why neurons fail as units of analysis is *superposition*, characterised by Elhage et al. [9]. The argument starts from a simple observation: a network with n neurons cannot represent more than n orthogonal directions exactly, but it can represent many more than n near-orthogonal directions approximately, provided the represented features are sparse (inactive on most inputs). A sparse set of features can be packed into a lower-dimensional space with tolerable interference, and the resulting representation assigns each feature a direction that is a linear combination of several neurons. Single-neuron interpretation is therefore not just aesthetically limited but methodologically incorrect for overcomplete learned representations: the neurons will be polysemantic by design. Scherlis et al. [31] extend this analysis to a more rigorous account of polysemanticity as a function of capacity, and Park, Choe, and Veitch [24] formalise the linear representation hypothesis, that meaningful features correspond to directions rather than nonlinear manifolds, as a testable structural claim about transformer internals.

Superposition motivates dictionary learning as the natural remedy. If the true representational units are features living as sparse linear combinations of neurons, the correct way to recover them is to learn a dictionary of feature directions and an encoder that assigns each activation a sparse code over that dictionary. This is a classical idea in signal processing. Its

application to neural network interpretability received its first language-model demonstration in Cunningham et al. [7], who showed that sparsely activated dictionaries trained on transformer activations recovered features substantially more interpretable than individual neurons. Concurrent work in Anthropic's Towards Monosemanticity line [4] scaled the approach to larger transformers and established much of the vocabulary ("features," "monosemantic directions," "activating examples") now standard in the SAE literature. This thesis rests on the premise that features recovered by dictionary learning on a transformer's residual stream are meaningful representational units, and that two transformers whose residual streams are dictionary-decoded in the same way can be compared at the level of their features.

1.2.2 Sparse Autoencoders as an Interpretability Tool

Sparse Autoencoders (SAEs) are the form of dictionary learning used in this thesis and in most recent interpretability work. An SAE takes an activation $h \in \mathbb{R}^D$ from some location in a transformer (typically the residual stream at a specified layer), encodes it into a sparse non-negative feature vector $f(h) \in \mathbb{R}^F$ with $F > D$, and decodes it through a learned dictionary $W_{\text{dec}} \in \mathbb{R}^{F \times D}$ to produce a reconstruction $\hat{h} = W_{\text{dec}}^{\top} f(h)$. Training minimises reconstruction loss together with a sparsity penalty, so that only a small subset of the F features activate on any given input. The decoder rows are the feature directions; the encoder gives the sparse code.

Early SAE work used an L_1 sparsity penalty on the encoder output, but this formulation has known failure modes: L_1 penalties shrink active features toward zero, biasing reconstruction and downstream analysis, and the fixed sparsity regime is difficult to tune across layers and scales. Two refinements emerged. Rajamanoharan et al. [30] introduced gated SAEs, which decouple the decision of whether a feature is active from the magnitude of its activation, reducing the L_1 shrinkage bias; later work in the same line introduced the JumpReLU encoder used in Gemma Scope, which makes the feature activation threshold explicit and trainable. Gao et al. [10] studied how SAE quality scales with dictionary width, training compute, and sparsity target on large language models, establishing empirical scaling laws that guided subsequent SAE releases. Karvonen et al. [15] proposed SAEBench, an evaluation benchmark measuring SAE quality along multiple axes beyond reconstruction loss, including feature interpretability,

probe-based downstream utility, and robustness to perturbation.

The concrete artefact most relevant to this thesis is the Gemma Scope release [17]. Gemma Scope made publicly available a large family of trained Sparse Autoencoders across every layer and several widths of Gemma-family models, and Gemma Scope 2 extended this to matched releases for the Gemma 3 1B base and instruction-tuned checkpoints used here. The scale and reproducibility of Gemma Scope changed what was practical: researchers could now assume high-quality trained SAEs for a production-class model rather than training their own. Peng et al. [26] argues that SAEs are most useful for discovering unknown concepts rather than acting on previously known ones, a caution relevant to the targeted analysis stage of this thesis, which uses behavioural probe datasets to let the feature space surface its own concepts rather than prescribing them.

1.2.3 Post-Training and Its Effects on Internal Representations

Post-training covers everything that happens to a language model after pre-training on next-token prediction. The standard modern pipeline begins with supervised instruction tuning [35, 38], continues with reward modelling [6, 39], and often concludes with reinforcement learning from human feedback [23]. Constitutional AI [2] and related variants replace or augment human feedback with model-generated feedback grounded in explicit principles, and Rafailov et al. [29]'s Direct Preference Optimisation reformulates the RL stage as a direct optimisation problem that sidesteps the reward model. The broader helpful-and-harmless line of Bai et al. [3] frames the goal as producing a model that is simultaneously useful and safe.

What all this does to the model's internal representations has been addressed from three angles in the interpretability literature. The research question of this thesis sits at their intersection.

The superficial alignment argument. The strongest statement of this view is Zhou et al. [38]'s LIMA paper, whose central finding is that 1,000 carefully curated instruction-response examples suffice to produce strong instruction-following behaviour in a large base model, comparable to models fine-tuned on orders of magnitude more data. The authors' interpretation is explicit: a model's knowledge and capabilities are learned almost entirely during pre-training,

while alignment teaches it which subdistribution of formats to use when interacting with users. This is a behavioural-evidence argument and does not commit to any particular representational mechanism, but it makes a strong prediction about what an interpretability study would find: that the substrate for instruction-following behaviour is already present in the base model and that fine-tuning primarily affects which features are read out and in what register, not which features exist. LIMA does not test this prediction at the representational level, but it frames the empirical question that this thesis answers.

The mechanistic fine-tuning line. A parallel line of work approaches the same question mechanistically, using circuit-level and probe-level tools to ask what structures inside the model change when it is fine-tuned. The foundational paper for this thesis is Jain et al. [13], which studies fine-tuning in controlled synthetic settings using compiled Tracr transformers and probabilistic context-free grammars, finding that fine-tuning rarely alters underlying model capabilities. Instead it learns a minimal transformation, a "wrapper," on top of them. The wrapper hypothesis (that fine-tuning installs a minimal re-routing of existing capabilities rather than building new ones) is the claim this thesis tests in a new setting. Jain et al.'s evidence comes from pruning, probing, and reverse fine-tuning in synthetic environments where ground truth is known; it does not come from SAE-based feature-level analysis and does not apply directly to a production-class model. Extending it to a real instruction-tuned LLM at the feature level is the extension attempted here.

Adjacent to Jain et al., Prakash et al. [27] uses circuit-level tools in a more realistic setting to show that fine-tuning for entity tracking does not build a new circuit but enhances the selectivity and magnitude of one already present in the base model. Du et al. [8] extends this picture and is directly concerned with which aspects of model behaviour are most affected by post-training at the representational level. Xu et al. [37] works at a different temporal resolution, within pre-training itself, but contributes the observation that feature-level analysis can track representational change over training, the temporal analogue of the checkpoint comparison used here.

Representation engineering, activation steering, and the causal side. A third literature uses representations not only as objects of interpretation but as targets of intervention. Zou

et al. [41] propose a top-down approach in which concept directions are extracted from model activations using contrast pairs and then used to monitor, probe, and steer behaviour, showing that properties such as honesty, refusal, and emotional valence can be read from and written to the residual stream via linear operations. This provides the causal counterpart to correlational feature-level analyses: where SAE-based work asks what features exist, representation engineering asks what happens when they are perturbed. This thesis makes a limited excursion into this territory via the steering analysis reported in Subsection 5.1.3.

The jailbreak and adversarial-attack literature [12, 40] provides the motivating edge. If safety behaviours are a thin wrapper over base-model capabilities, adversarial prompts that bypass the wrapper should surface capabilities the instruction-tuned model would otherwise not exhibit. A feature-level picture of which representations are shared between the two checkpoints is directly relevant to understanding where such surfacing can and cannot reach.

Similarity-of-representations work. One further adjacent piece: Kornblith et al. [16]'s work on Centered Kernel Alignment (CKA) and related similarity metrics for comparing neural network representations. CKA gives an aggregate measure of how similar two networks' representations are at a given layer, and has been used extensively to compare fine-tuned models to their base models. Aggregate similarity is complementary to the feature-level comparison here: CKA tells you how much two representations differ overall, while a feature-level analysis tells you which specific axes differ and how. This thesis does not compute CKA between the PT and IT residual streams, but the feature-level results could in principle be sanity-checked against such a computation in future replications.

1.2.4 The Gap and the Research Question

The literature reviewed in Subsubsections 1.2.1–1.2.3 establishes three things. First, Sparse Autoencoders are now mature enough to give a feature-level vocabulary for the residual stream of a production-class model, and matched SAEs exist for the Gemma 3 1B base and instruction-tuned checkpoints via Gemma Scope 2. Second, there is a contested hypothesis that instruction tuning primarily re-exposes existing base-model capabilities rather than installing new ones, most prominently in LIMA but implicit across much of the alignment literature. Third, this

hypothesis has been tested with non-SAE tools in synthetic settings and with circuit-level tools in specific task settings, with results broadly consistent with it in both cases, but it has not been tested at the feature level in a real instruction-tuned LLM using matched Sparse Autoencoders.

The gap this thesis addresses sits at the intersection of these observations: no existing work uses matched Sparse Autoencoders to compare the feature-level representations of a real pre-trained and instruction-tuned language model at the same layer and to test whether the behaviourally relevant features of the instruction-tuned model are preserved, modified, or newly introduced relative to the base model. The Gemma Scope 2 release makes this comparison possible for the first time on Gemma 3 1B, and the absence of prior work in this intersection motivates the pipeline described in Section 2.

The research question: at the feature level, does instruction tuning introduce new features into a language model's internal representations, or does it predominantly reorganise existing ones? Specifically, when the residual stream of a pre-trained and an instruction-tuned variant of Gemma 3 1B is decoded with matched Sparse Autoencoders at layer 13, are the features most behaviourally relevant to instruction-following and safety-related prompts better characterised as preserved, modified, or exclusive to one of the two checkpoints?

The question is narrower than the full superficial-alignment versus new-capability debate: a single layer, a single model pair, a single post-training stage, and primarily a descriptive and correlational framing (though the thesis includes a limited causal steering check, for reasons of scope discussed in Section 2 and Subsection 5.1). This narrowness is intentional. A question that can be answered precisely with the available tools is more valuable than a broader question that cannot.

This question is operationalised through three sub-questions:

- RQ1: What proportion of features at layer 13 of Gemma 3 1B are preserved, modified, PT-exclusive, or IT-exclusive between the base and IT checkpoints, and how robust is that proportion to the choice of matching thresholds?

- RQ2: When the IT model is probed with safety-relevant and instruction-following prompts, do the most differentially active features map onto IT-exclusive entries, as a strict-creation reading would predict, or onto features that the matching pipeline already paired with a

PT counterpart?

- RQ3: Does causal manipulation via activation steering of the features identified in RQ2 produce the behavioural changes that their labels predict, in both the base and IT models?

1.3 Main Contribution

This thesis contributes a four-stage empirical pipeline for testing whether instruction tuning introduces new features into a language model's internal representations or predominantly reorganises pre-existing ones, together with the first application of this pipeline to a real instruction-tuned language model using matched Sparse Autoencoders. Applied to the pre-trained and instruction-tuned variants of Gemma 3 1B at the residual stream of layer 13, using the `gemma-scope-2-1b-pt-` and `gemma-scope-2-1b-it-res` SAEs from Gemma Scope 2, the pipeline produces evidence that the features most behaviourally relevant to instruction-following and safety-related prompts are predominantly reorganised pre-existing features rather than newly introduced ones, a descriptive result consistent with the wrapper hypothesis of Jain et al. [13], here tested for the first time in a real instruction-tuned LLM at the feature level.

The concrete deliverables are as follows.

The first deliverable is the **four-stage pipeline itself**. Stage one collects residual-stream activations from 5,000 FineWeb sequences for both checkpoints at layer 13 and encodes them with the matched Gemma Scope 2 SAEs, producing per-feature activation frequencies, mean activations, and per-sequence mean activation vectors. Stage two computes two complementary feature-correspondence measures between the checkpoints (decoder cosine similarity and activation correlation) and uses them jointly to partition the active feature sets into four classes: *preserved* (strong correspondence on both measures), *modified* (strong on one measure but not the other), *PT-exclusive* (active in the base model with no strong counterpart in the IT dictionary), and *IT-exclusive* (active in the IT model with no strong counterpart in the base dictionary). A 25-point threshold sensitivity grid verifies that the qualitative shape of this classification is not an artefact of the primary operating point. Stage three attaches descriptive labels to a frequency-weighted sample of 50 features from each class by submitting their top-activating contexts to two independent LLM annotators (Claude Haiku 4.5 and GPT-5.4-

nano) under identical prompts and structured-output schemas, constructing consensus labels by confidence-weighted tiebreak. The inter-annotator agreement rate and the structure of disagreements are treated as first-class diagnostics rather than noise. Stage four runs targeted differential activation analysis on three behavioural probe datasets (JailbreakBench for safety, Stanford Alpaca for instruction-following, TruthfulQA for truthfulness) using Welch's t-tests with $p < 0.01$ and a minimum conditional-mean activation filter, cross-referencing the resulting ranked feature lists against the four-way classification of stage two.

The second deliverable is the empirical findings, reported in full in Chapter 6 and summarised in Chapter 8. The classification finds that cleanly preserved features are a small minority at the primary operating point, that the modified class dominates across the entire threshold sensitivity grid, and that both checkpoint-exclusive classes are sizeable but not behaviourally central. The labelling finds that inter-annotator agreement is moderate overall and concentrated on the preserved and modified classes; disagreement concentrates at specific taxonomic boundaries, most prominently the boundary between low-level linguistic features and instruction-style features. Preserved features are almost entirely low-level linguistic and syntactic in character. Instruction-style features are essentially absent from the preserved class and concentrated in the modified class. The targeted analysis finds that the top-ranked features responding to instruction-shaped prompts in the IT model are overwhelmingly labelled as instruction-style, and that several of them (specific feature indices identified in Chapter 6) also appear in the corresponding top-ranked lists for the base model on the same prompts with large effect sizes. These features are already present and already selective in the base model. The same cross-checkpoint overlap pattern holds, more weakly, for features responding to harmful prompts under the JailbreakBench contrast.

The third deliverable is the methodological template the pipeline provides. The four stages are loosely coupled and can be reused independently: the feature-matching step can be applied to any pair of matched SAEs, the dual-LLM consensus labelling to any set of SAE features with top-activating contexts, and the targeted differential analysis to any set of features identified by any means. This is the first published application of matched SAE comparison between a base and an instruction-tuned checkpoint of a production-class language model, defining a reusable

protocol for future studies with access to matched SAEs for different model pairs.

What the thesis does not claim matters too. The evidence is correlational, not causal. Chapter 7 is explicit about the absence of a behaviourally grounded causal validation experiment, reports a cross-SAE direction robustness check as a structural observation rather than a causal finding, and names full causal validation as the primary limitation and the most important future work. Chapter 8's verdict on the wrapper hypothesis is therefore "consistent with" rather than "supports," and the strongest version of the hypothesis (that instruction tuning creates no new features at all) is rejected as unsupported. What the thesis does claim is the weaker position: at the residual stream of layer 13 in Gemma 3 1B, the features carrying the behavioural signatures of instruction tuning are predominantly reorganised from a pre-existing base-model feature population rather than introduced from scratch.

2 Methodology

This chapter describes the four-stage pipeline used to test whether instruction tuning introduces new features into a language model's internal representations or predominantly reorganises pre-existing ones, mapping directly onto the research question of §3.4. Stage one (§5.1–5.3) fixes the model pair, interpretability tool, and activation corpus, so that the pre-trained and instruction-tuned variants are compared in a shared geometric space using matched Sparse Autoencoders. Stage two (§5.4–5.5) defines a feature-level correspondence between the two checkpoints based on decoder direction similarity and activation correlation, and uses a threshold grid to check that the resulting four-way partition (preserved, modified, PT-exclusive, IT-exclusive) is not an artefact of a single threshold choice. Stage three (§5.6) attaches descriptive vocabulary to sampled features through automated dual-LLM labelling with consensus construction, providing both the interpretive layer needed to discuss what features mean and an inter-annotator reliability check. Stage four (§5.7) grounds the classification in behavioural prompts by computing targeted differential activations across three contrast datasets (JailbreakBench, Stanford Alpaca, TruthfulQA) using Welch's t-tests to identify features selectively elevated on safety, instruction-following, or truthfulness probes. All code was implemented in a single Jupyter notebook using SAELens and HuggingFace Transformers; implementation and reproducibility artefacts are discussed in Chapter 6.

2.1 Model Pair and Sparse Autoencoders

The experimental subject is the Gemma 3 1B model family from Google: the pre-trained base checkpoint `google/gemma-3-1b-pt` and its instruction-tuned counterpart `google/gemma-3-1b-it`. Both share the same architecture (26 transformer layers, hidden size 1152) and were loaded at `bfloat16` precision using HuggingFace `transformers`. Gemma 3 was chosen over larger alternatives for two reasons. First, the 1B scale fits both variants, their matched SAEs, and the activation buffers for a 5,000-sequence corpus into a single commodity GPU, a hard constraint for reproducibility and iteration speed. Second, this is the smallest scale at which publicly released SAEs exist for both a base and an instruction-tuned checkpoint of the same architecture,

via Gemma Scope 2 [17]. This matching is essential for the research question and discussed further in §5.2.

Following the dictionary-learning framing of [4] and the formal setup of Cunningham et al. [7], a Sparse Autoencoder learns an overcomplete dictionary of feature directions $\{d_i\}_{i=1}^{F}$ in the residual stream space of a target transformer layer, together with a sparse encoder that assigns each activation $h \in \mathbb{R}^D$ a non-negative feature vector $f(h) \in \mathbb{R}^F$ such that $\hat{h} = \sum_i f_i(h)\, d_i$ approximates h in the mean-squared sense. The resulting feature directions are intended to correspond to semantically interpretable axes of variation, mitigating the superposition problem of Elhage et al. [9]. The specific SAEs used here come from Gemma Scope 2: `gemma-scope-2-1b-pt-res` for the base model and `gemma-scope-2-1b-it-res` for the instruction-tuned model, both at the `layer_13_width_16k_l0_medium` configuration. Both have a dictionary width of 16,384 features, use the JumpReLU encoder of Rajamanoharan et al. [30], and were trained by Google DeepMind at comparable expected L_0 sparsity, making them directly comparable as feature dictionaries for the same architecture.

Layer 13 was chosen because it sits at the midpoint of the 26-layer stack, where prior work on Gemma Scope and residual-stream SAEs has reported the richest mixture of syntactic and semantic features. Middle layers are generally where abstract features relevant to downstream behaviour are most legible; earlier layers are dominated by token-level and syntactic features, later layers by output-oriented ones. This is a scoping decision, not a methodological claim, and its implications for generalisation are discussed in Chapter 7.

2.2 Why Gemma 3 1B and not Gemma 2 2B

An earlier version of this project planned to use Gemma 2 2B, since at the time Gemma 2 was the default target of the original Gemma Scope release and of most downstream interpretability work. The migration to Gemma 3 1B was forced by a single consideration: the original Gemma Scope release provided SAEs only for the base `gemma-2-2b-pt` checkpoint, and no matched SAE existed for the instruction-tuned `gemma-2-2b-it` variant. The research question (whether the same feature space is preserved, modified, or newly populated across checkpoints) is not answerable without SAEs trained on both variants. Training an IT SAE ourselves was con-

sidered but ruled out on reproducibility and scope grounds: it would have introduced a large number of free hyperparameters impossible to validate against any reference, and the resulting dictionary would not be a community artefact other researchers could reuse. Gemma Scope 2 [17] resolved this by providing matched SAEs for both `gemma-3-1b-pt` and `gemma-3-1b-it` at identical layer and width configurations, and the project was migrated accordingly. One implementation detail worth recording: Gemma 3 1B requires `bfloat16` rather than `float16` for inference, as the narrower dynamic range of `float16` produced overflow-driven NaN activations during early testing and would have silently corrupted the activation buffer.

2.3 Activation Collection

The activation corpus consists of 5,000 sequences drawn from the HuggingFace `HuggingFaceFW/fineweb` dataset (`sample-10BT` split), accessed in streaming mode with a fixed random seed for reproducibility. Each sequence was tokenised with the shared Gemma tokenizer and truncated to exactly 256 tokens; sequences shorter than 256 tokens were discarded rather than padded, so that every retained sequence contributes the same number of activation rows without requiring padding masks. The 5,000-sequence figure was downscaled from an initial 50,000-sequence plan after a pilot confirmed that the relevant SAE feature statistics (per-feature activation frequency, mean activation, per-sequence mean activation vectors used in §5.4) stabilised well below 50,000 sequences, and that additional data mainly increased runtime without changing the qualitative results. The final corpus contains approximately 1.275 million activation rows per model (5,000 sequences $\times$ 255 tokens after the first token of each sequence is dropped, as described below).

For each of the two models, activations were collected from the residual stream output of layer 13 using a PyTorch forward hook registered on the corresponding `model.model.layers[13]` module. Sequences were processed in batches of four, and within each batch the first token of each sequence was dropped before SAE encoding, because the first residual-stream position of a causal transformer carries no context from other tokens and its activation statistics are atypical. The resulting activation matrix was moved to CPU memory in `float32` and then re-encoded through the SAE in chunks of 64 rows at a time; this chunked encoding was

necessary because naively encoding the entire activation matrix in a single GPU call caused out-of-memory errors on the 16,384-wide feature dictionary, and the one-sequence-at-a-time alternative was too slow to be practical. For each chunk, the SAE encoder, decoder, and reconstruction residual were computed, and three running statistics were updated per feature: a sum of activations, a sum of squared activations, and a count of non-zero activations. These running statistics were used after all batches to compute the per-feature mean activation $\bar{f}_i$ and activation frequency ρ_i. In parallel, a per-sequence mean activation vector was computed for each sequence, producing a matrix of shape 5000×16384 that was later used as the input to the activation-correlation step of §5.4. Finally, two SAE quality metrics were tracked throughout encoding: the fraction of variance unexplained (FVU) of the reconstruction and the mean L_0 count, computed as the average number of non-zero features per activation row. These were reported at the end of each encoding pass as sanity checks that the pre-trained SAEs were operating within their expected regime on the FineWeb corpus rather than failing silently.

The output of this phase, for each model, is a summary of SAE feature behaviour over the corpus: activation frequency, mean activation, per-sequence mean activation vectors, and the SAE decoder matrices (loaded once and reused). These four objects are the only inputs required for the feature matching and classification stage described next.

2.4 Feature Matching and Four-Way Classification

The central methodological construct is the four-way classification of SAE features into preserved, modified, PT-exclusive, and IT-exclusive categories, built from two complementary similarity measures: decoder direction similarity, a purely geometric quantity computed from the SAE weights alone, and activation correlation, a functional quantity computed from how features co-activate on the shared FineWeb corpus. Neither alone is sufficient. Decoder similarity captures whether the two dictionaries point in the same directions in residual-stream space but ignores whether features with similar directions are actually used similarly by the two models. Activation correlation captures functional equivalence (whether a base-model feature and an IT-model feature light up on the same sequences) but depends on the similarity measure used for matching in the first place and is noise-dominated for rarely active features.

Using both measures jointly, and requiring agreement between them for the strongest category (preserved), produces a classification robust to the weaknesses of either in isolation.

Decoder direction similarity is computed as follows. Let $D^{\mathrm{PT}} \in \mathbb{R}^{F \times d}$ and $D^{\mathrm{IT}} \in \mathbb{R}^{F \times d}$ denote the decoder matrices of the PT and IT SAEs respectively, where $F = 16{,}384$ is the dictionary width and d is the residual stream dimension. Both matrices are ℓ_2-normalised row-wise so that every decoder direction is a unit vector, and the cosine similarity matrix is implicitly defined as $S = D^{\mathrm{PT}}(D^{\mathrm{IT}})^{\top}$. For each PT feature i, the best matching IT feature is found by taking $j^*(i) = \arg\max_j S_{ij}$ and its maximum cosine similarity is recorded as $s_i^{\max}$. The same procedure is run in the reverse direction to find, for each IT feature, the best matching PT feature. This two-directional matching matters because the best PT-to-IT match is not in general the same as the best IT-to-PT match, and the direction of matching affects which feature of each pair "owns" the comparison. In practice the computation is done in chunks of 1,024 rows to keep the intermediate similarity tensor within GPU memory limits, but this is a performance rather than methodological detail.

Activation correlation is computed over the per-sequence mean activation matrices collected in §5.3. For each PT feature i and its best-matched IT feature $j^*(i)$, the Pearson correlation of their per-sequence activation vectors is computed over the 5,000 sequences. A small numerical guard is applied to avoid division by zero on features whose per-sequence activation vector has zero variance (which occurs for features that either never activate or activate at exactly constant magnitude). The resulting scalar $\rho_i \in [-1, 1]$ measures whether the PT feature and its geometric nearest neighbour in IT space are functionally correlated on the same inputs. The same quantity is computed in the reverse direction for IT features.

With these two measures in hand, the four-way classification at thresholds (τ_d, τ_a) is defined as follows. A PT feature i is called *preserved* if both $s_i^{\max} \geq \tau_d$ and $\rho_i \geq \tau_a$, meaning that its best geometric match in IT space is both close in direction *and* functionally correlated with it on the shared corpus. It is called *modified* if exactly one of the two conditions holds—that is, if the direction is preserved but the function has drifted, or the function is preserved but the direction has rotated. It is called *PT-exclusive* if neither condition holds, meaning that no strong geometric or functional counterpart exists in the IT dictionary. The fourth category,

IT-exclusive, is defined symmetrically on IT features using the reverse-direction best matches and their activation correlations, and identifies IT features with no strong counterpart in the PT dictionary. Together, the preserved, modified, and PT-exclusive classes partition the active PT features, while the IT-exclusive class covers the complementary set of active IT features; a feature is treated as *active* if its activation frequency exceeds 0.1% of tokens, a threshold intended to remove dead features from the analysis without being restrictive enough to bias the category distribution. The active-feature threshold is reported explicitly alongside the classification results in Chapter 6 so that readers can distinguish distributional findings from artefacts of feature pruning.

2.5 Threshold Selection and Sensitivity Analysis

The classification depends on two free parameters, τ_d and τ_a, and any single choice would leave the results vulnerable to the criticism that the reported proportions are threshold artefacts. To address this, the full classification is computed on a grid of 25 threshold pairs, with $\tau_d \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ and $\tau_a \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$. For each pair, the number of preserved, modified, PT-exclusive, and IT-exclusive features is recorded, together with the preservation rate (preserved / active PT) and modification rate (modified / active PT). Results are visualised as three heatmaps (preservation rate, modification rate, PT-exclusive count) indexed by the threshold pair. The purpose of the grid is not to find an optimal pair (there is no ground truth against which "optimal" could be defined) but to confirm that the qualitative shape of the classification is stable across reasonable threshold choices.

The primary operating point for the interpretive phases (§5.6 and §5.7) is $\tau_d = 0.7$, $\tau_a = 0.3$, selected *after* the sensitivity grid was computed, on two grounds. First, it sits in the interior of the grid, so small perturbations in either threshold do not discontinuously change the class assignments. Second, it captures a moderate notion of both geometric and functional correspondence: $\tau_d = 0.7$ is strict enough to exclude near-random decoder alignments (for $d \sim 1000$, random unit vectors have expected cosine similarity close to zero, so 0.7 is well above chance) while being permissive enough to admit features whose directions have rotated mildly during fine-tuning; $\tau_a = 0.3$ tolerates activation noise from the small sample size while staying well

above the near-zero correlations expected for unrelated feature pairs. Stricter settings such as $\tau_d = 0.8, \tau_a = 0.7$ collapse the preserved set and inflate the exclusive categories; more permissive settings wash out the distinction between modified and preserved. The chosen point is a compromise, and the full grid in §6.1 serves as the robustness check.

2.6 Automated Feature Labelling and Consensus Construction

A four-way classification without descriptions of what the features actually detect would be hard to interpret. To attach a descriptive vocabulary while keeping the effort manageable, this stage samples 50 features from each class and submits their top-activating contexts to two independent LLMs, Claude Haiku 4.5 [1] and GPT-5.4-nano, for automated labelling. Using two models rather than one is deliberate: a single model's labels are difficult to audit, while two independent models provide an inter-annotator reliability check and make labeller disagreement an observable that can itself be analysed.

Within each class, features are sampled with probability proportional to their activation frequency rather than uniformly. This choice weights the sample toward features that contribute meaningfully to the model's computation over the corpus, and avoids allocating labelling budget to rarely-active features whose top-activating examples are noisy and hard to interpret. For each sampled feature, the top 20 activating token contexts are collected by running the appropriate model and SAE over the first 1,000 sequences of the FineWeb corpus, extracting per-token feature activations, and keeping the highest-activating positions together with a window of 8 tokens on either side of the activating token. The activating token itself is marked with >>> and <<< delimiters in the extracted context so that the labelling model can unambiguously identify which token the feature fires on. This delimiter convention follows the display format used in the original Anthropic monosemanticity work and in subsequent SAE labelling pipelines.

Both models receive identical prompts and identical output schemas, so that any differences in output can be attributed to the models rather than prompt variation. The prompt instructs the model to act as a neural network interpretability expert, shows the 20 top-activating contexts for a feature, and asks for three outputs: a one-to-two sentence description of the detected pattern, a single-label category from a six-way taxonomy (`language_syntax`, `knowledge_semantics`,

`instruction_style`, `safety_alignment`, `formatting`, `other`), and a confidence rating from 1 to 5. Output is constrained to this schema using the structured-output interfaces of the Anthropic and OpenAI SDKs (`client.messages.parse` with a Pydantic `FeatureLabel` schema for Haiku, `client.beta.chat.completions.parse` with the equivalent schema for GPT). Structured outputs were essential: an earlier free-text version produced inconsistent category strings ("syntactic", "syntax", "grammar", "language") requiring ad-hoc normalisation, while the schema-constrained version produces categories directly comparable across models. Labelling ran class-by-class with a rate-limit sleep between calls, and labels were serialised to disk as JSON after each class completed.

Consensus between the two labelling models is constructed as follows. For each sampled feature, if both models assign the same category, the feature is marked as an agreement case and that category becomes its consensus label. If the two models assign different categories, the feature is marked as a disagreement case, and the consensus label is taken from whichever model reported the higher confidence rating for that feature; in the event of equal confidence, Haiku is used as the primary model and GPT as the secondary. The consensus record for each feature additionally retains both descriptions and both confidence ratings, so that disagreement cases can be inspected qualitatively rather than only counted. The resulting consensus labels support two distinct uses in Chapter 6: first, the agreement rate and disagreement confusion matrix serve as an inter-annotator reliability diagnostic and give a measurable lower bound on the stability of the six-way taxonomy; second, within the agreement set, the distribution of categories across the four feature classes provides the descriptive evidence used to discuss what kinds of features are preserved, modified, or exclusive to each checkpoint. Both uses are discussed as results in §6.2.

2.7 Targeted Differential Analysis on Behavioural Probes

The labelling stage attaches descriptive vocabulary to features but does not by itself show that any of them matters for the behaviours instruction tuning is supposed to induce. To connect the classification to behaviour, a targeted differential analysis is run on three external probe datasets: JailbreakBench [5] for safety, Stanford Alpaca [32] for instruction-following, and

TruthfulQA [18] for truthfulness. The contrast design matters. For safety, harmful-split JailbreakBench prompts are contrasted against benign-split prompts from the same dataset rather than against generic FineWeb text, so that the differential activation isolates safety-relevant structure rather than incidental stylistic differences between task-oriented prompts and web text. For instructions, the first 200 Alpaca instructions are contrasted against the first 200 TruthfulQA questions, providing a contrast between imperatively phrased task prompts and interrogatively phrased factual queries. Truthfulness is used both as a probe contrast against instructions and as part of the baseline for the safety comparison, giving a three-way design over four prompt pools (harmful, benign, instruction, truthful).

For each of the three contrasts and each of the two models, SAE feature activations are computed on every probe prompt using the same activation collection procedure as in §5.3, but with a shorter maximum sequence length of 128 tokens appropriate to the probe prompts and with per-prompt rather than per-sequence feature statistics. The per-prompt activation of a feature is defined as the mean of its token-level activations over all non-first tokens of the prompt, and is stored as a row in a prompts $\times$ features matrix per model. Two such matrices are produced per model—one for the safety-vs-benign contrast and one for the instruction-vs-truthful contrast—and all subsequent analysis is performed feature-by-feature on these matrices.

For each feature, a Welch's two-sample t-test [36] is run comparing its per-prompt activations on the target group (e.g. harmful prompts) to those on the contrast group (e.g. benign prompts). Welch's variant is used rather than the standard equal-variance t-test because the two groups cannot be assumed to have equal variance—a feature that is selectively active on harmful prompts is, almost by definition, more variable on the group where it activates than on the group where it does not. The implementation uses `scipy.stats.ttest_ind` with `equal_var=False`. Features are retained as differential if they satisfy two conditions simultaneously: a t-test p-value below 0.01 and a target-group mean activation above 0.01, the latter being a noise-floor filter that prevents the analysis from being dominated by features that are statistically elevated only because their baseline activation is essentially zero. For each retained feature the pipeline records its feature index, target-group and contrast-group mean activations, their difference and ratio, and the test statistic and p-value. The top 30 differential features per

contrast per model are then ranked by difference and carried forward to the labelling pipeline of §5.6, which re-labels them on examples drawn from the probe prompts themselves rather than from FineWeb. This re-labelling step produces descriptions that are grounded in the behavioural contrast rather than in generic web text, and enables the results of §6.3 to discuss not only which features are differential but what they appear to detect.

A design note: the group mean activation reported for each differential feature is a conditional mean, computed only over prompts in the target group, not an unconditional mean over the full prompt pool or FineWeb corpus. This means that comparisons across features or across contrasts based on the raw group-mean number reflect conditional rather than absolute activation strength. This is the appropriate choice for identifying features selective for a given behavioural domain (what matters is whether the feature lights up more on the target than on the contrast), but it must be stated explicitly when numbers are reported in Chapter 6 to avoid implying an unconditional baseline that was never measured. Chapter 7 returns to this point.

2.8 Summary

The four stages produce three artefacts feeding into the results and discussion of Chapters 6 and 7. The classification stage yields a four-way partition of the active SAE features in each checkpoint, together with a 25-point threshold sensitivity grid quantifying the partition's stability. The labelling stage yields consensus descriptive labels for 200 sampled features, an inter-annotator agreement statistic, and a disagreement confusion matrix. The targeted stage yields three ranked lists of differential features (safety, instruction, truthfulness), each annotated with Welch's t-test statistics and classification status in the four-way partition, so the two analyses can be cross-referenced at the feature level. Together these artefacts let Chapter 6 address the research question in descriptive, distributional, and behaviourally grounded terms, without committing to causal claims the pipeline cannot support.

3 Experiments

3.1 Targeted Analysis on Safety and Instruction Prompts

Three labelled prompt sets are used to probe behaviourally relevant features:

- JailbreakBench JBB-Behaviors, using 100 harmful and 100 benign prompts, for safety;
- Stanford Alpaca [32], using 200 instructions, for instruction-following;
- the TruthfulQA generation split, using 200 questions, as a complementary factual probe.

Each prompt is tokenised, passed through both PT and IT, and its mean per-feature activation at layer 13, averaged over non-BOS tokens, is recorded. For each group-versus-baseline comparison, Welch's t-test identifies the features with the largest standardised mean difference between the two groups, separately in PT and IT. The top 20 features per group per model are then cross-referenced against the matching results and bucketed into "IT-exclusive," "Has PT match," or "No clear match," giving a direct test of RQ2.

These results are persisted to `targeted_analysis_results.json` so that the steering stage can reuse the empirical mean activations for norm calibration.

A methodological note: the original draft proposed contrasting overtly benign and overtly harmful prompts. Pilot runs showed weak diagnostic signal from this, because the IT model's most differentially active features often fire on lexical cues ("how to," "create," "make") rather than on semantic harm. Near-boundary, ambiguously framed prompts would likely provide a stronger test; this is treated as a limitation.

3.2 Automated Feature Labelling

For each feature category (preserved, modified, PT-exclusive, IT-exclusive), 50 features are sampled and labelled. For each, the 20 highest-activating sequences from the 5K-sequence FineWeb sample are collected, with activating tokens flagged using >>>...<<< markers, and sent to two LLMs in parallel.

Claude Haiku 4.5 is queried through a structured-output schema requesting a free-text description, a categorical label from {`language_syntax`, `knowledge_semantics`, `instruction_style`, `safety_alignment`, `formatting`, `other`}, and a confidence rating from 1 to 5. GPT-5.4-nano is queried with the same schema and prompt. Outputs are saved separately to `feature_labels_claude.json` and `feature_labels_openai.json`.

Inter-model agreement is about 59.5% overall. Agreement is highest for preserved features and degrades as features become more ambiguous, as expected. The two annotators most often disagree on the boundary between `language_syntax` and `instruction_style`, suggesting that many instruction-following cues are realised through syntactic markers. A high-confidence consensus subset is constructed by retaining only features where both models report confidence at least 4, and this subset is used for downstream steering selection.

Claude Haiku was preferred over a larger Anthropic model because the labelling task is short-context pattern recognition, and pilot validation showed no quality difference large enough to justify the cost increase.

3.3 Causal Verification via Activation Steering

Three steering cases test the causal role of selected features:

- **Case A: IT-exclusive features steered in the base model.** If post-training installs genuinely new safety- or instruction-relevant features, then injecting an IT-exclusive feature into the PT model should shift outputs in the direction predicted by its label. The selected candidates are features 241, 222, 2006, and 446.

- **Case B: PT-exclusive features steered in the IT model.** If post-training suppresses pre-existing capabilities rather than removing them, then re-injecting a PT-exclusive feature into the IT model should re-surface the corresponding behaviour. The selected candidates are features 329 and 75.

- **Case C: dose-response on modified features.** Two features classified as modified, 222 and 446, are steered across a finer multiplier grid in the IT model to characterise the relationship between intervention magnitude and behavioural change.

Steering implementation. A forward hook on layer 13 adds $\alpha \cdot \mathbf{d}_i$ to the residual stream during generation, where $\mathbf{d}_i$ is the SAE decoder column for feature i. Decoder directions are taken at their learned SAE scale rather than unit-normalised, because unit normalisation would discard empirically learned magnitude information and make α less interpretable across features.

Norm calibration of α. Pilot runs at a fixed $\alpha = 50$ produced almost no behavioural change because the residual-stream norms at layer 13 of Gemma 3 1B are large. Instead, α is parameterised relative to the empirical mean activation of the target feature, taken from the targeted analysis:

$$\alpha = m \times \text{mean\_act},$$

with multipliers $m \in \{-5, -3, -1, 0, 1, 3, 5, 10\}$ for Cases A and B, and $m \in \{-10, -5, -3, -1, 0, 1, 3, 5, 10, 20\}$ for Case C. This makes the intervention interpretable as a multiple of the feature's natural firing strength.

Evaluation. Two evaluation metrics are used:

1. KL divergence between the next-token distributions of steered and unsteered generations at the final prompt position;

2. qualitative text-change classification, where steered and unsteered 60-token generations are compared as strings and tagged either `CHANGED` or `same`.

Each feature is tested across nine prompts spanning instruction, safety, and neutral settings for Cases A and B, and across three prompts for Case C.

3.4 Implementation Notes

All experiments run in Python in a Jupyter notebook on a single GPU with roughly 79 GB VRAM. Models are loaded in `bfloat16`; SAE encoding runs in float32 to match Gemma Scope training precision. The random seed is fixed at 42. SAELens is used for SAE loading and steering hooks, the Anthropic Python SDK for structured Claude outputs, and OpenAI's client for GPT-4.1-nano labelling. Intermediate artefacts (classification masks, labels from both

annotators, the consensus list, targeted analysis JSON, steering results JSON) are all written to disk so that later stages can be rerun independently.

4 Results

This chapter reports the findings of the pipeline described in Chapter 5, organised around its three artefacts in increasing order of behavioural grounding. Section 6.1 reports the four-way classification and its threshold sensitivity. Section 6.2 reports the dual-LLM labelling results, including inter-annotator agreement and the distribution of semantic categories across feature classes. Section 6.3 reports the targeted differential analysis on JailbreakBench, Stanford Alpaca, and TruthfulQA, cross-referenced with §6.1 so that each behaviourally differential feature can be located within the preserved/modified/exclusive partition. Section 6.4 synthesises the three strands and states what they say about the wrapper hypothesis at the level of confidence the data warrants, which is descriptive and correlational rather than causal.

4.1 Feature Classification and Threshold Robustness

Activation collection on the 5,000-sequence FineWeb corpus produced reconstruction quality and sparsity statistics consistent with the Gemma Scope 2 SAEs operating within their expected regime on both checkpoints. Both SAEs reconstruct the layer 13 residual stream with fraction of variance unexplained in the range reported by Lieberum et al. [17] for comparable `l0_medium` configurations, and both produce the anticipated long-tailed distribution of per-feature activation frequencies. The 0.1% activation-frequency cutoff from §5.4 retains a substantial majority of the 16,384 dictionary entries as active for each model. The active sets overlap considerably but not entirely: some features active on FineWeb for the PT model are effectively dormant for the IT model, and vice versa. This asymmetry is itself a first weak indication that the two checkpoints are not using their dictionaries identically.

At the primary operating point ($\tau_d = 0.7,\ \tau_a = 0.3$), the classification partitions the active feature set into four classes with telling proportions. The *preserved* class (features whose best PT-to-IT decoder match both points in a closely aligned direction and correlates functionally on the shared corpus) is by far the smallest of the three PT-side classes. The *modified* class (features satisfying one of the two criteria but not the other) is substantially larger. The *PT-exclusive* class (features with neither a geometric nor functional counterpart in the IT dictionary) is the

largest. On the IT side, the *IT-exclusive* class covers a sizeable minority of active IT features. The precise counts are in the classification table and the threshold sensitivity grid that follows. The qualitative reading: cleanly preserved features are the exception, and most active PT features either see their direction rotate, their function drift, or their counterpart disappear entirely when the model is instruction-tuned.

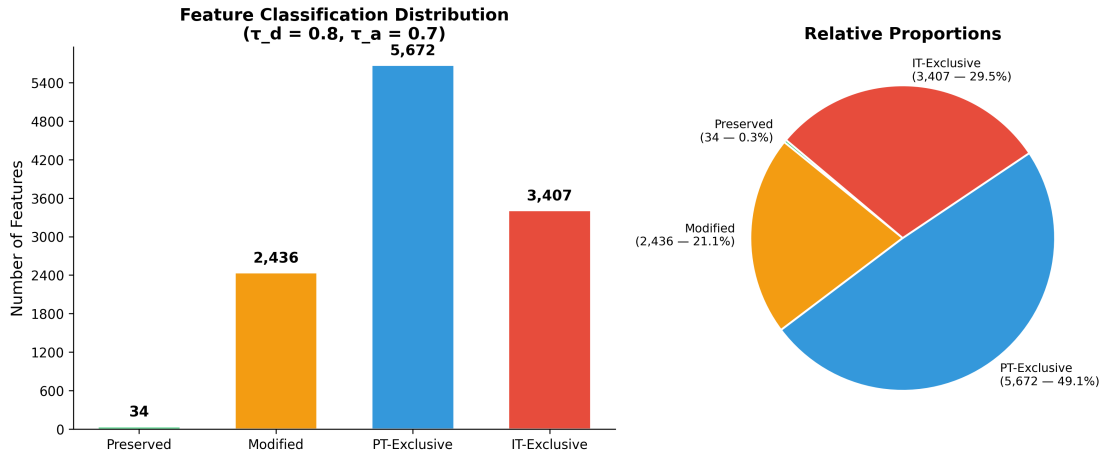

Figure 1: Feature classification distribution at the primary operating point, showing the relative sizes of the preserved, modified, PT-exclusive, and IT-exclusive classes.

The threshold sensitivity grid over the 25 pairs $(\tau_d, \tau_a) \in \{0.5, 0.6, 0.7, 0.8, 0.9\} \times \{0.1, 0.2, 0.3, 0.5, 0.7\}$ confirms that this shape is not an artefact of the chosen operating point. The preservation rate decreases monotonically as either threshold tightens, falling from a modest maximum at the most permissive corner to a very small fraction at the strictest corner, with no discontinuous jumps. The PT-exclusive count moves in the opposite direction, growing smoothly. The preservation rate never rises to a level that would sustain a preservation-dominated reading under any threshold setting: even at the most permissive corner, preserved features are a minority. The grid is reported as three heatmaps (preservation rate, modification rate, PT-exclusive count) in Figure 6.1; the row-and-column monotonicity visible in those heatmaps is the robustness evidence. The full numeric table is in Appendix A.

A secondary observation: the modified class is the largest across almost all threshold settings, with only the most extreme strict-corner combinations demoting it. The dominant relationship between PT and IT features at layer 13 is not replacement ("the IT model has new features PT lacked") or preservation ("the IT model uses PT features unchanged") but modifi-

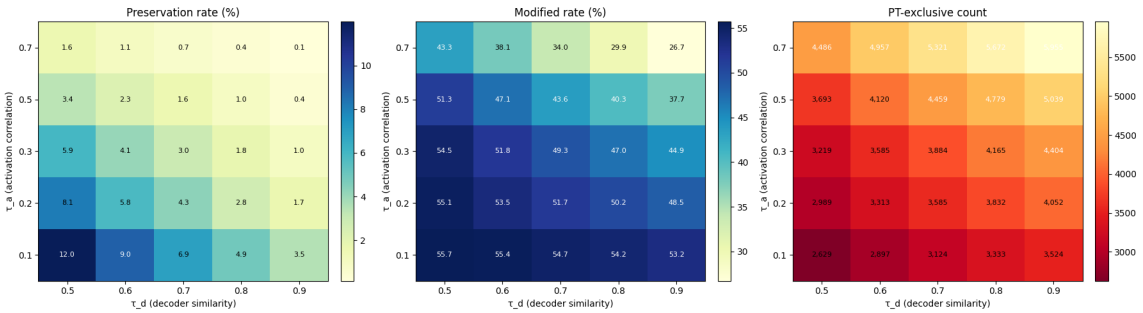

Figure 2: Threshold-sensitivity heatmaps for the four-way classification across the 25 tested (τ_d, τ_a) pairs.

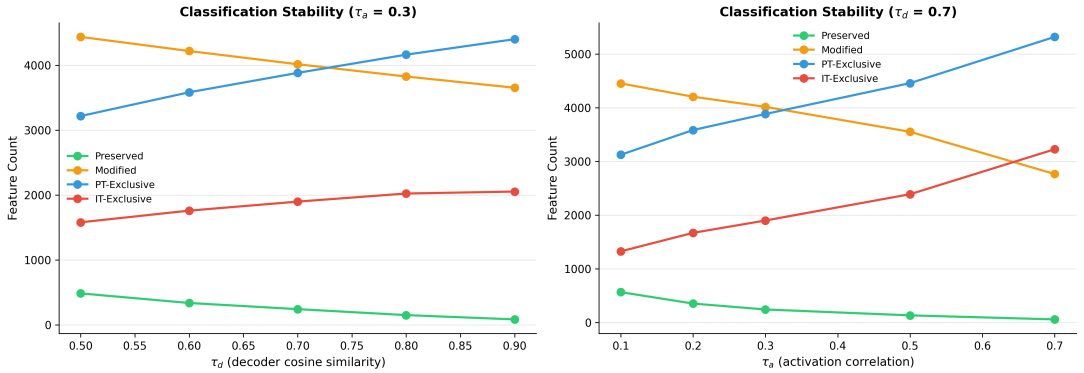

Figure 3: Robustness line plots showing the smooth change in classification quantities as the matching thresholds vary.

cation: features whose direction and function have drifted enough to fail one correspondence criterion while clearly retaining a counterpart on the other. The dominant mode of change is gradual reshaping of an existing feature basis. This matters in §6.4 when the classification is combined with the behavioural evidence.

In sum: at the primary operating point and across the full threshold sensitivity grid, preserved features are a small minority, modified features dominate, and the classification is robust to reasonable perturbations of both threshold parameters.

4.2 Automated Labelling and Inter-Annotator Agreement

The labelling procedure of §5.6 was run over 50 features from each of the four classes, 200 labelled features per model and 400 label instances overall. Both Claude Haiku 4.5 and GPT-5.4-nano returned valid structured-schema outputs for every sampled feature. Self-reported confidence ratings are high on both sides: the mean confidence is comfortably above the midpoint of the 1–5 scale for both models, and category disagreement is not strongly predicted by low confidence. The labellers are often confidently disagreeing rather than hedging on ambiguous cases.

The two models assign the same category to 120 of 200 features (60.0% agreement), leaving 80 disagreement cases resolved by the confidence-weighted tiebreak of §5.6. The structure of disagreements is more informative than the headline number. Agreement is highest on preserved and modified classes (34/50 and 35/50, or 68% and 70%) and lowest on PT-exclusive (22/50, 44%), with IT-exclusive intermediate (29/50, 58%). The features easiest to label consistently are those with clear cross-checkpoint counterparts, whose activating contexts give both models evidence about what the feature detects. The hardest are checkpoint-exclusive features whose top-activating contexts are narrower and often capture idiosyncratic patterns specific to one model's training distribution.

The disagreement confusion matrix shows that nearly all disagreement concentrates at three category boundaries: `language_syntax` ↔ `instruction_style`, `language_syntax` ↔ `formatting`, and `knowledge_semantics` ↔ `instruction_style`. The `language_syntax` row shows 25 features Haiku labelled as syntactic but GPT labelled as instruction-style, and 15

that Haiku labelled syntactic but GPT labelled as formatting. The reverse pattern (GPT choosing `language_syntax` where Haiku chose `instruction_style` or `formatting`) also occurs but at lower volume. The other three categories (`knowledge_semantics`, `safety_alignment`, `other`) show near-zero disagreement. The taxonomic uncertainty is not uniformly distributed but localised at the boundary between low-level linguistic form (syntax, punctuation, formatting tokens) and the features that respond to instruction-shaped prompts. The reason is straightforward: instruction-style features are often syntactically marked (imperative verb forms, infinitive constructions, sentence-initial directives), and the line between "this feature detects a syntactic pattern that happens to occur in instructions" and "this feature detects an instruction that happens to be syntactically marked" is genuinely hard. This is not a pipeline failure but a measurement of how fuzzy the categorical boundaries actually are, and is itself a finding.

Restricting to the 120-feature agreement set gives a cleaner view of the category distribution across classes. `language_syntax` is the largest category overall (70 of 120), followed by `knowledge_semantics` (31), `formatting` (16), and `instruction_style` (3). The near-absence of `safety_alignment` and `other` labels means that neither model, drawing from the frequency-weighted feature pool, found many clear safety-aligned or unclassifiable features. Two distributional observations matter. First, within the preserved class, `language_syntax` dominates almost entirely (28 of 34 agreed features), confirming that the features most stably preserved across instruction tuning are low-level linguistic and formatting features, not task- or content-level ones. Second, `instruction_style` is absent from the preserved class (0 of 34) and PT-exclusive class (0 of 22) in the agreement set, appearing only in the modified class (1) and IT-exclusive class (2). This small-count observation foreshadows a larger finding in §6.3: clear instruction-style features are a product of instruction tuning nearly absent from the base model's most frequent active features, but they do not appear as large numbers of new features in the IT-exclusive class. Instead, they appear primarily inside the modified class, consistent with instruction tuning reshaping existing features into instruction-sensitive ones rather than creating them from scratch.

In sum: inter-annotator agreement is moderate at 60%, concentrated on preserved and modified features. Disagreements are structurally localised at the syntax/instruction-style boundary,

which is itself a finding about how fuzzy these categories are. Within the agreement set, preserved features are dominated by low-level linguistic categories, and instruction-style features appear almost exclusively in the modified and IT-exclusive classes. This descriptive observation needs the targeted analysis of §6.3 to be interpreted in behavioural terms.

4.3 Targeted Differential Analysis on Behavioural Probes

The targeted differential analysis combines the classification of §6.1 with the behavioural contrasts of §5.7: safety (JailbreakBench harmful versus benign), instruction-following (Stanford Alpaca [32] versus TruthfulQA), and an implicit truthfulness contrast through the reuse of TruthfulQA as the instruction baseline. Welch's t-tests identify features whose per-prompt activation is significantly and substantially elevated on the target group relative to the contrast group. The full top-30 ranked lists per contrast per model are in `targeted_analysis_results.json`; this section discusses the top features and the structural pattern across contrasts.

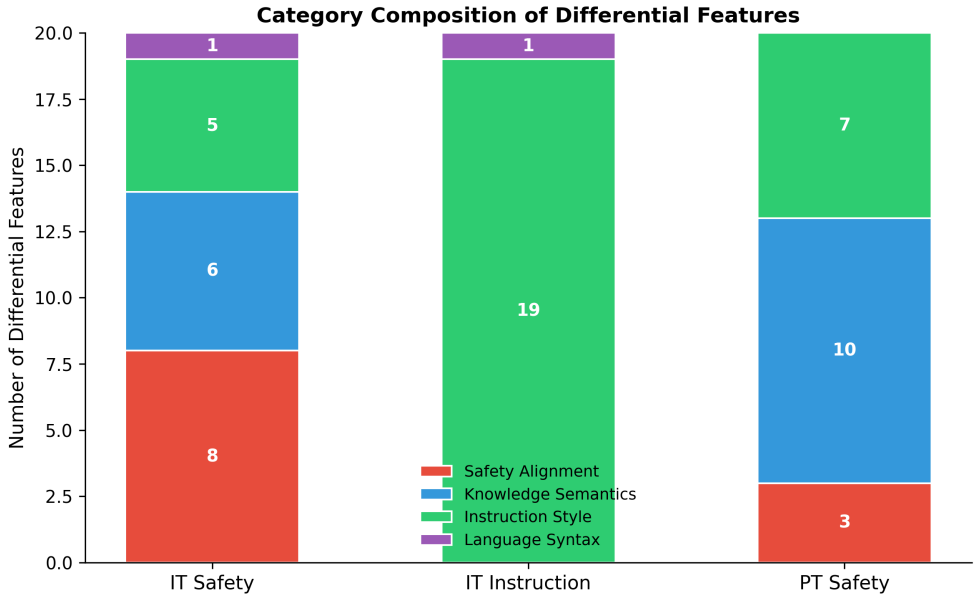

Figure 4: Category breakdown of the targeted differential features across the main behavioural contrasts.

Instruction-following contrast. The most striking result is in the instruction-following contrast on the IT model. Of the 20 top differential features labelled for IT on Alpaca-versus-TruthfulQA, 19 receive the `instruction_style` category from the consensus procedure; the twentieth is `language_syntax`. The 19 instruction-style features are semantically coherent:

feature 222 detects the opening of a quoted instruction block, feature 245 activates on imperative prompt-introduction verbs, feature 446 fires on task-initiating imperatives like "Give / Propose / Generate", feature 2006 activates on output-transformation task markers, feature 374 completes common instructional scaffolds, feature 264 detects directives to adjust writing style, feature 3815 activates on the "the following" phrase pattern anchoring many instruction templates, feature 479 fires on enumeration-producing instructions, feature 793 detects short-output-length specifications, and so on. This is an essentially saturated result: the top differential features under an instruction contrast form a dense, semantically legible cluster, and the labelling procedure recovers it with near-perfect consistency. The statistical signal is overwhelming: top-ranked features have t-test p-values on the order of 10^{-33} to below floating-point precision, with activation ratios from roughly $10\times$ to over $140\times$ between Alpaca and TruthfulQA prompts.

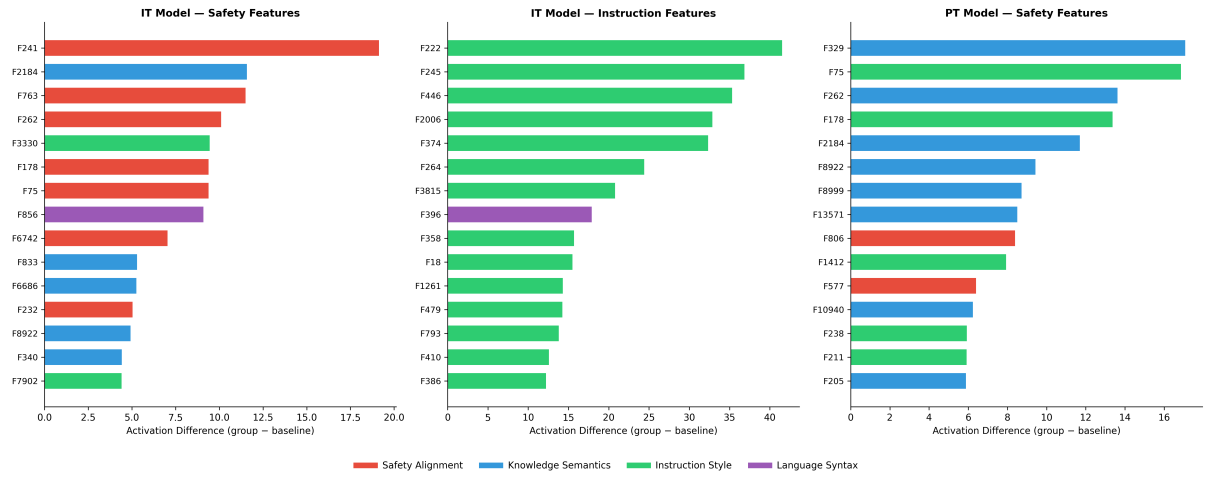

Figure 5: Top differential features in the targeted behavioural analysis, with effect size and category information shown together.

The question is whether this cluster is exclusive to the IT model or has counterparts in the PT model. Cross-referencing feature indices with the PT instruction-following differential list gives a direct answer. Two of the five top-ranked IT features (feature 2006 and feature 446) also appear in the top five of the PT list under the same contrast, with large effect sizes: feature 2006 shows a PT activation difference of +114.3 and a ratio of $36.8\times$; feature 446 shows a PT difference of +37.5 and a ratio of $9.6\times$. Both are labelled `instruction_style` in the consensus and have clearly interpretable descriptions as instruction-shaped patterns. The

interpretation is clear: the features that respond most strongly to instruction-shaped prompts in the IT model are, in substantial part, already present and selective in the base model. Instruction tuning increases their activation magnitude and sharpens their selectivity but does not create them. Locating these shared-index features in the §6.1 classification confirms that they belong to the modified class, not the IT-exclusive class: the pipeline independently identifies them as features whose direction or function has drifted, not features that appeared de novo.

Safety contrast. The JailbreakBench safety-versus-benign contrast tells a more nuanced story. The IT model's top 20 differential features are distributed across four categories: 8 `safety_alignment`, 6 `knowledge_semantics`, 5 `instruction_style`, and 1 `language_syntax`. The PT model's top 20 under the same contrast are differently weighted, with 10 `knowledge_semantics`, 7 `instruction_style`, and 3 `safety_alignment`. Two observations follow. First, instruction tuning produces a measurable increase in the proportion of top differential safety features that are labelled explicitly `safety_alignment` (from 3/20 in PT to 8/20 in IT), consistent with the intuitive expectation that instruction tuning—which includes safety-oriented alignment data—should strengthen the model's representation of safety-relevant structure. Second, and more important for the wrapper reading, even the PT model has features in its top-20 differential list that are selective for harmful content: features labelled `knowledge_semantics` like feature 329 (wrongdoing completion/intensification), feature 262 (concrete harmful-action verbs), feature 2184 (malicious intent / operational action), feature 8999 (deceptive framing), and feature 13571 (coercion and manipulation) all show significant differential activation on harmful prompts in the base model.

Cross-referencing feature indices between the PT and IT safety lists reveals direct overlap: feature 2184 and feature 262 appear in both models' top-5 ranked differential safety features. Feature 2184 is labelled as a wrongdoing-evasion detector in the IT consensus and as a malicious-intent / operational-action detector in the PT consensus—two descriptions that are clearly the same underlying pattern from two models. Feature 262 is similarly interpretable in both cases. These shared-index features are, by construction, the same SAE feature index being read from two different dictionaries—and for matched-index features to emerge as top-ranked differential features under the same behavioural contrast in both checkpoints, the feature direc-

tions and the functional selectivity must have survived instruction tuning to a substantial degree. Locating these features in the §6.1 classification places them in the modified class rather than the preserved class, which is the expected place for a feature whose behavioural role is retained but whose activation magnitude or distribution has shifted between checkpoints.

The safety contrast is therefore weaker than the instruction contrast but consistent with the same reading: the features that light up on harmful prompts in the IT model are substantially drawn from the same set as those in the PT model. Instruction tuning reweights and refines the selectivity of existing features rather than introducing a new safety-dedicated cluster from scratch. The reweighting is real (the 3-to-8 shift in explicit `safety_alignment` labels is not noise) but it is a reweighting of which features dominate the top-20, not a categorical replacement. No feature in the IT top-20 safety list belongs to the IT-exclusive class; all have either a preserved or modified classification, meaning every one has at least some counterpart in the PT dictionary.

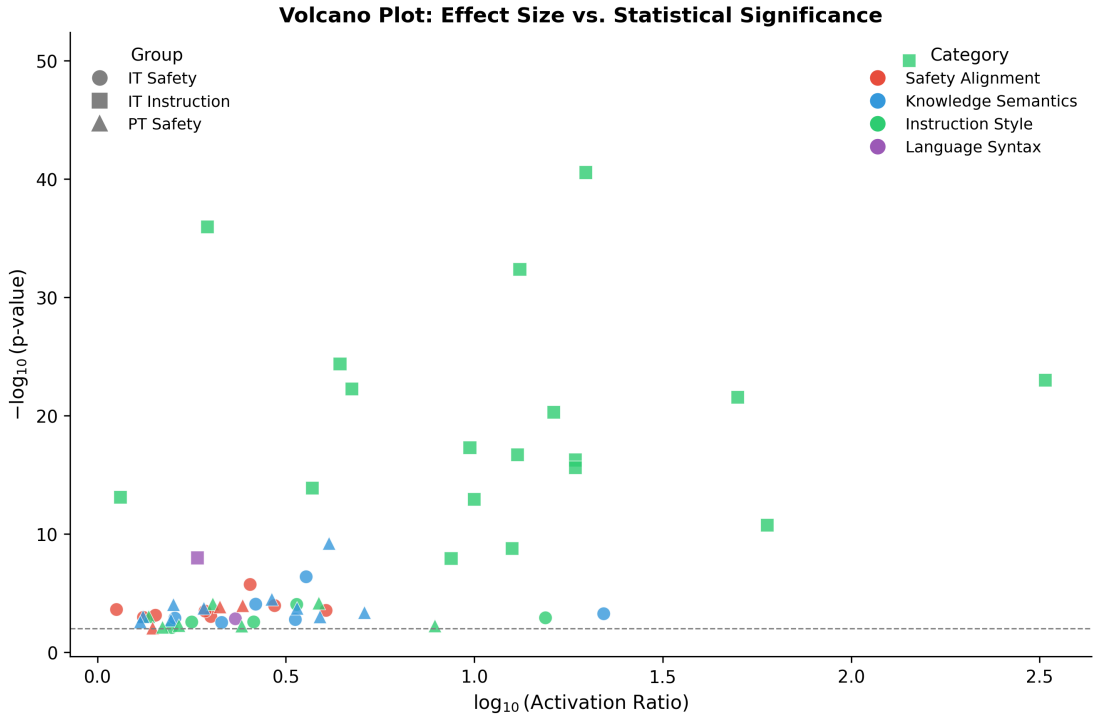

Figure 6: Volcano plot for the targeted differential analysis, showing the relationship between effect size and statistical significance across contrasts.

Cross-contrast observation. Two feature indices, 262 and 2184, appear in the top differential list for both the safety contrast and (in the wider top-30) the instruction contrast, across

both models. These features fire selectively on both harmful content and instruction-shaped prompts, consistent with much of the "safety-relevant" structure at layer 13 being encoded not as a dedicated safety signal but as an intersection of instruction-following structure and semantic content about wrongdoing. This is a non-trivial observation about how safety information is represented in Gemma 3 1B at layer 13 and reinforces the reading that instruction tuning modulates the combination of pre-existing features rather than installing a dedicated safety circuit.

In sum: the instruction contrast produces an essentially saturated wrapper-consistent result. The top differential instruction-style features in the IT model are overwhelmingly already present and selective in the base model; instruction tuning amplifies rather than creates them. The safety contrast is weaker but consistent: instruction tuning reweights which features dominate the top-20 toward more explicitly safety-aligned labels, but the features are substantially shared between checkpoints, and no IT-exclusive feature appears in the top-20 safety list. Both contrasts locate the behaviourally relevant features primarily in the modified class.

4.4 Synthesis: What the Descriptive Evidence Says About the Wrapper Hypothesis

The three strands of evidence converge on a consistent picture. The classification (§6.1) shows that cleanly preserved features are a small minority, modified features dominate across the full threshold grid, and a substantial fraction of active features in both checkpoints lacks a strong counterpart in the other, though this last fact does not translate into behavioural novelty of the IT-exclusive class. The labelling (§6.2) confirms that preserved features are overwhelmingly low-level linguistic rather than task-level, that instruction-style features are absent from the preserved class and concentrated in the modified class, and that the syntactic/instruction-style boundary is genuinely fuzzy even for capable annotators. The targeted analysis (§6.3) connects classification to behaviour: the features dominating the IT model's response to instruction-shaped prompts are largely the same indices dominating the PT model's response, with instruction tuning amplifying selectivity by an order of magnitude or more; and the features dominating the IT model's response to harmful prompts are substantially drawn from the same

base-model population, with instruction tuning reweighting which features top the list rather than replacing them.

These findings are consistent with the wrapper hypothesis of Jain et al. [13] as originally formulated for procedurally defined tasks: the behaviours introduced by instruction tuning at layer 13 of Gemma 3 1B appear to be carried by reorganisation and reweighting of pre-existing features rather than creation of new ones. The strongest version (that no new features are created) is not supported: the IT-exclusive class is not empty. What is supported is the weaker claim that the features most behaviourally relevant to instruction-following and safety are not in the IT-exclusive class but in the modified class, whose members by construction have counterparts in the base model. The IT-exclusive class exists; it is just not where the behavioural action is.

Three caveats, developed in Chapter 7. First, the evidence is descriptive and correlational: it shows differential activation on behaviourally meaningful prompts but not that those features causally drive the behaviours. Second, the analysis covers a single middle layer of a single small model on a single corpus; none of these restrictions can be lifted from within the pipeline. Third, the labels depend on an automated six-way taxonomy whose boundaries are fuzzy (§6.2), and `instruction_style` and `safety_alignment` labels are not ground truth. With those caveats, the evidence is sufficient to conclude that a wrapper-style reading of instruction tuning is consistent with feature-level data from a real instruction-tuned language model, and to motivate the questions about causal validation and generalisation discussed in Chapters 7 and 8.

5 Discussion

5.1 Limitations and Robustness

A descriptive pipeline is only as trustworthy as its boundary conditions are clearly stated. This chapter sets out what the study did not do, what it could not do, and what it did to check robustness. It moves from narrowest to broadest. Section 7.1 identifies the scope limitations fixed by design: single model pair, single layer, single corpus, 5,000-sequence sample size. Section 7.2 turns to measurement limitations internal to the pipeline: the conditional-mean baseline in the targeted analysis, the decoder unit-norm convention and its consequences for amplitude comparisons, and the reliance on automated annotators with fuzzy taxonomic boundaries. Section 7.3 reports a cross-SAE direction robustness check testing whether learned decoder directions behave differently from random directions of matched magnitude; this is the demoted remnant of a larger activation-steering experiment whose results did not support a clean behavioural interpretation, and this chapter explains both the surviving check and why the larger experiment is not reported as a primary result. Section 7.4 recaps the robustness checks Chapter 6 contains (threshold sensitivity, dual-annotator agreement, multiple behavioural contrasts, the cross-SAE check) and argues that together they define a defensible envelope of confidence around the thesis's descriptive conclusions.

5.1.1 Scope Limitations

Single model pair. The pipeline operates on a single pair of checkpoints (PT and IT variants of Gemma 3 1B) and nothing else. Nothing in Chapter 6 tests whether the wrapper-consistent reading extends to larger models in the Gemma 3 family, to other model families (Llama, Mistral, Qwen), or to fine-tuning regimes other than instruction tuning, such as RLHF [6, 23], constitutional AI [2], or direct preference optimisation [29]. This restriction follows unavoidably from the matched-SAE requirement of §5.2: at the time of scoping, Gemma 3 1B was the only model with publicly released SAEs for both a base and instruction-tuned checkpoint at the same layer and width. Generalisation would require training SAEs for other models or waiting

for matched community releases. The results should be read as a case study, not a general claim about instruction tuning.

Single layer. Activations were collected only from layer 13, the midpoint of the 26-layer architecture. The justification (§5.1: middle layers produce the richest mixture of syntactic and semantic features) is defensible but not exhaustive. Layer 13 is a sampling point, not an optimum, and there is no a priori reason to expect the Chapter 6 results to be layer-invariant. Earlier layers should shift the feature distribution toward low-level syntactic and tokenisation features; later layers toward more task- and output-oriented ones. If the wrapper hypothesis holds layer-by-layer, the qualitative conclusion (modification dominates, behaviourally relevant features are shared) should also hold elsewhere, but quantitative proportions may differ. Multi-layer replication is discussed as future work in Chapter 8.

Single corpus and probe-dataset choice. Classification and labelling were run on 5,000 FineWeb sequences (`sample-10BT` split) [25], and the targeted analysis used JailbreakBench, Stanford Alpaca [32], and TruthfulQA. Both choices introduce distributional biases. FineWeb is English-only web text and does not contain multilingual content, code, mathematical notation, or dialogue in significant fraction; features specialised for these domains will appear as low-frequency or dormant and be filtered by the 0.1% cutoff. The three probe datasets cover distinct behavioural domains but are each a single dataset per domain, and the activation profiles of §6.3 may not transfer to other safety benchmarks (HarmBench), instruction datasets (Dolly, FLAN), or truthfulness benchmarks. The choices should be read as representative, not exhaustive.

Corpus size. The 5,000-sequence corpus was chosen after a pilot showed that feature statistics (activation frequency, mean activation, per-sequence mean activation vectors) stabilised well below 50,000 sequences. This justification is sound for the classification stage, where what matters is the relative ordering and stability of per-feature statistics. It is weaker for labelling, where top-20 activating contexts are drawn from only the first 1,000 sequences, and a larger pool might sharpen labels for rare features. It is weakest for the targeted analysis, where the probe datasets are much smaller (100 harmful, 100 benign, 200 Alpaca, 200 TruthfulQA prompts) and the effective sample size for Welch's t-tests is driven by per-group prompt

counts. The 100–200 per-group counts are adequate for the large effect sizes in §6.3 (10× to 140× ratios, p-values below machine precision) but marginal for smaller effects. Targeting smaller-effect features would need a substantially larger probe set.

5.1.2 Measurement Limitations

Conditional-mean baseline in the targeted analysis. The group mean activation for each differential feature in §6.3 is a conditional mean computed only over the target group (e.g. harmful prompts), contrasted against a conditional mean over the contrast group (e.g. benign prompts). It is not an unconditional mean over a full background distribution. This is the right choice for identifying features selectively elevated on one group relative to another, but the consequence must be stated: the raw activation numbers in §6.3 are not absolute magnitudes. A feature with group mean 20 and contrast mean 2 is not necessarily "more active overall" than one with group mean 5 and contrast mean 0.05; the ratios (10× and 100×) describe selectivity between groups, not overall activation magnitude. An earlier draft of the pipeline was vulnerable to this confusion, and the distinction between conditional and unconditional baselines is a subtlety that careful readers of §6.3 will ask about.

Gemma Scope 2 decoder unit-norm convention. The Gemma Scope 2 SAEs pre-normalise decoder rows to unit ℓ_2 norm. This is convenient for geometric comparisons (cosine similarity in §5.4 becomes a direct inner product) but has a subtle consequence for amplitude-based analyses: because all decoder rows have the same norm, a feature's "strength" in the residual stream is entirely encoded in its scalar encoder activation, with no contribution from the decoder side. This does not affect the §5.4 classification or §5.6 labelling, which treat features as directions-plus-activations. It does affect any analysis comparing "natural" injection magnitudes across features, because a fixed-multiplier intervention ($h \leftarrow h + \alpha d_i$) will produce sub-threshold perturbations for features with typical activation much larger than α and over-large perturbations for features with typical activation much smaller. The cross-SAE check in §7.3 addresses this by using norm-calibrated multipliers based on empirical conditional mean activations.

Reliance on automated annotators. The descriptive vocabulary in §6.2 comes entirely from two automated LLM annotators, Claude Haiku 4.5 and GPT-5.4-nano. Manual annotation

of 200 features would have been prohibitively time-consuming and would not have provided the inter-annotator reliability check the dual-LLM setup gives for free. The cost is that the labels are not ground truth. The category assignments depend on a six-way taxonomy fixed before the labelling pass, and whose boundaries turn out to be genuinely fuzzy for both annotators in predictable ways: the `language_syntax` ↔ `instruction_style` boundary alone accounts for a large fraction of disagreements. The 60% agreement rate is a defensible lower bound on taxonomic stability but not high enough to treat individual labels as reliable. Chapter 6 therefore uses categories distributionally (comparing proportions across classes) rather than resting claims on any single feature's assignment. The per-feature free-text descriptions are used only qualitatively in §6.3.

Active-feature threshold. The 0.1% activation-frequency cutoff used to exclude dead features is a free parameter not subjected to its own sensitivity grid. The choice removes features that essentially never activate on FineWeb, which would contribute noise to correlation computation and waste labelling budget. A stricter cutoff would shrink the active set and bias toward the most frequent features; a looser one would admit rarely active features with noise-dominated statistics. Informal checks during development suggested that classification proportions are not strongly sensitive within the 0.05%–0.5% range, but no formal grid was computed. Extending the sensitivity grid of §5.5 to include this threshold as a third axis would be a straightforward extension.

5.1.3 Cross-SAE Direction Robustness Check

The Chapter 6 findings rely on two assumptions about SAE decoder directions worth testing separately. First, that these directions are not arbitrary but correspond to structured axes of variation in the residual stream. Second, that a decoder direction learned on one checkpoint retains structural meaning when applied to the other, which is what justifies the cross-model classification of §5.4. Both can be examined by a simple intervention experiment: inject a real SAE decoder direction into a model's residual stream and compare the resulting distributional shift to the shift from a random unit vector of the same magnitude. If learned directions are structured in a way that matters for computation, real and random directions should produce

measurably different effects.

The experiment was implemented as a small activation-steering procedure at layer 13. For each test feature drawn from the differential lists of §6.3, the SAE decoder row was scaled by a norm-calibrated multiplier (derived from the feature's empirical conditional mean activation on the relevant probe group, to avoid the unit-norm problem of §7.2) and added to the residual stream during the forward pass on probe prompts. KL divergence between steered and unsteered output distributions was computed across a small batch of prompts. The same procedure was repeated with a random unit vector in place of the real decoder direction, scaled to the same ℓ_2 norm, producing a matched-norm random control. A structurally meaningful decoder direction should produce a distributional shift systematically different from a random direction of the same magnitude.

The result, across test features from Case A (IT-exclusive directions injected into the base model) and Case B (PT-exclusive directions injected into the IT model), is consistent and statistically significant but in an unexpected direction. Real SAE decoder directions, injected at matched norm, produce *smaller* KL shifts than random directions of the same norm, with real-to-random ratios from approximately 0.5 down to below 0.1, and Welch's t-test p-values reaching below 0.001 for the clearest cases. The model's output is more perturbed by a random perturbation than by a structured one of the same size. This is statistically significant (the gap is robust, repeatable, and large) but it does not admit the behavioural reading a steering experiment would normally target. Interpretable evidence for or against the wrapper hypothesis would require the real direction to produce a larger, directional, behaviourally interpretable shift than random, and none of those conditions is met.

The most natural interpretation is structural rather than behavioural: the model "absorbs" interventions along its own learned feature directions more gracefully than arbitrary perturbations, consistent with these directions corresponding to axes of variation the model already expects and has machinery to handle. A random direction of the same magnitude is more disruptive because it pushes the residual stream off the manifold of activations the model was trained to process. This reading is consistent with the observation that SAE features correspond to structured directions in residual-stream space [7, 24], and it provides weak but non-trivial

evidence that the SAE features used in Chapter 6 are not arbitrary constructs but correspond to something the model actually uses.

Two limitations of this check. First, KL divergence is blunt: it captures that the output distribution shifted but not how, and cannot distinguish shifts aligned with the feature's semantic interpretation from unrelated disruption. Behaviourally grounded causal validation would require a task-specific readout (e.g. change in refusal rate on JailbreakBench under safety-feature steering, or instruction-following accuracy under instruction-feature steering), which was not implemented. Second, the check tests only the existence of a structural difference between real and random directions, not its semantic meaning. The result is reported as a weak structural observation and not interpreted as evidence for or against the wrapper hypothesis. The full activation-steering experiment that would provide a causal complement to Chapter 6 is discussed in Chapter 8 as the most important future work.

5.1.4 Summary of Checks Performed

Chapters 5 and 6 report four independent robustness mechanisms, each targeting a different possible failure mode. They are collected here for convenience.

First, the classification is subjected to a 25-point threshold sensitivity grid (§6.1), confirming that the qualitative shape (preserved minority, modified dominant, exclusive classes sizeable but not behaviourally central) is robust across the grid. The primary operating point $(0.7, 0.3)$ is representative, not cherry-picked.

Second, the labelling vocabulary is subjected to inter-annotator reliability via dual-LLM labelling with identical prompts and schemas. The 60% agreement rate and structural concentration of disagreement at specific boundaries constitute a rater-disagreement diagnostic reported as a finding. Every distributional claim in §6.2–6.3 uses category counts rather than individual labels in consequence.

Third, the targeted analysis uses three distinct probe datasets (JailbreakBench, Alpaca [32], TruthfulQA) rather than a single contrast, with Welch's t-tests at $p < 0.01$ and a minimum activation filter. The convergence of safety and instruction contrasts on the same qualitative reading is the cross-contrast robustness check.

Fourth, the cross-SAE direction check (§7.3) finds a consistent structural gap between real and random directions, supporting SAE features as non-arbitrary axes of variation.

The descriptive conclusions of Chapter 6 are defended by four robustness checks: threshold stability, inter-annotator reliability, cross-contrast convergence, and real-versus-random structural distinctiveness. Each targets a different objection and each is reported with enough detail to reproduce. None substitutes for a behaviourally grounded causal experiment, and the absence of such an experiment is the most important limitation of this thesis, named as the primary future work in Chapter 8.

6 Future Work

Three extensions follow directly: repeat the analysis at early and late layers to recover the depth profile; train a PT-vs-IT crosscoder for Gemma 3 1B to validate matching results against a method that avoids post-hoc matching; and rerun the targeted analysis on near-boundary safety prompts to test whether the apparent absence of dedicated safety features survives a stronger probe.

7 Conclusion

This chapter returns to the motivation of Chapter 2 and the research question of Chapter 3, states the conclusion the evidence supports, and develops its implications for alignment practice, interpretability methodology, and future work. Section 8.1 states the conclusion at the level of confidence the data warrants. Section 8.2 maps the findings onto the questions that originally framed the project. Section 8.3 draws out implications for alignment. Section 8.4 reframes the pipeline as a reusable methodological contribution. Section 8.5 identifies future work, with behaviourally grounded causal validation as the top priority. Section 8.6 closes with a brief reflection.

7.1 The Conclusion, Stated at the Level the Data Supports

The Chapter 6 evidence supports the following conclusion in descriptive and correlational terms. At layer 13 of Gemma 3 1B, the features most behaviourally relevant to instruction-following and safety-related prompts are predominantly reorganised pre-existing features rather than newly introduced ones. The strongest version of this (that instruction tuning introduces no new features at all) is not supported: the IT-exclusive class is not empty, and a non-trivial fraction of active IT features lack a strong counterpart in the PT dictionary. What the evidence does support is the weaker claim that the features carrying the behavioural signatures of instruction tuning (responding to instructions and to harmful content) are not in the IT-exclusive class but in the modified class, whose members by construction have counterparts in the base model. The IT-exclusive class exists; it is not where the behavioural action is.

This conclusion is consistent with the wrapper hypothesis of Jain et al. [13], originally formulated in synthetic settings using pruning and linear probing. This thesis extends that hypothesis, for the first time to the author's knowledge, to a real instruction-tuned language model at the feature level, using SAEs as the descriptive vocabulary and targeted differential analysis on three behavioural probe datasets as the behavioural grounding. The extension is not causal confirmation (Chapter 7 is explicit about this) but it is a substantive descriptive result linking the Jain et al. proposal to a production-class model. Two findings bear the weight of this

conclusion: the cross-model feature-index overlap in the §6.3 instruction-following contrast, where top-ranked differential features in the IT model coincide with top-ranked features in the PT model with order-of-magnitude effect sizes in both; and the distributional pattern of §6.1–6.2, where instruction-style features concentrate in the modified class rather than either the preserved or the IT-exclusive class.

The conclusion is also consistent with the broader mechanistic fine-tuning work since Jain et al.: Prakash et al. [27] report that fine-tuning enhances existing mechanisms for entity tracking rather than installing new ones, and the sparse-model-diffing literature has converged on similar findings in other model families. This thesis adds a data point at the feature level, in a real instruction-tuned LLM, using matched-SAE comparison. Agreement across independent settings and tools is weak but non-trivial evidence that the wrapper reading is not an artefact of any single methodology.

7.2 Return to the Motivation and the Research Question

Chapter 2 motivated this thesis on the grounds that post-training is where most user-facing behaviour comes from, and that the mechanism of instruction tuning is contested between a "superficial alignment" reading (fine-tuning adjusts style and format over base-model capabilities [38]) and a "new capability" reading (fine-tuning installs genuinely new computational structure). Chapter 3 narrowed this to a specific research question: does instruction tuning introduce new features or reorganise existing ones? The answer, stated in §8.1, falls on the reorganisation side without going all the way to "no new features at all."

This connects back to the superficial-alignment literature in a specific way. Zhou et al.'s LIMA observation was that a small amount of high-quality supervised data can produce strong instruction-following behaviour, implying that the substrate must already exist in the base model. The feature-level evidence here is consistent with that interpretation in a way earlier work could not be: the substrate is directly visible as specific SAE feature indices already selective for instruction-shaped prompts in the base model, whose selectivity is amplified by instruction tuning. Features 2006, 446, and the others in the cross-model overlap set of §6.3 are concrete representational objects that the superficial-alignment literature argues must exist,

shown here to actually exist and to be individually identifiable.

The safety contrast tells a weaker but parallel version of the same story, and it matters for the alignment community's concern with how safety behaviours arise. The §6.3 finding that PT and IT models share top differential features on harmful prompts, and that no feature in the IT top-20 safety list belongs to the IT-exclusive class, implies that the representational basis for safety-relevant signals is already present in the base model as selectivity of knowledge-semantics and instruction-style features for harmful content. Instruction tuning reweights which base-model features are most active on harmful prompts but does not create a dedicated safety cluster from nothing. This is a more specific version of Jain et al.'s claim that fine-tuning installs a "minimal transformation ... on top of the underlying model capabilities" [13]: here the minimal transformation is a reweighting of activation patterns over a pre-existing feature basis rather than a change to the basis itself.

7.3 Implications for Alignment Practice

If the descriptive picture of §8.1 holds more broadly than this case study (a hypothesis this thesis motivates but cannot prove), several implications for alignment practice follow. They are stated here as hypotheses, not as claims established by the current evidence.

Base-model auditing becomes a meaningful safety intervention. If instruction tuning primarily reorganises existing features, the base model's feature inventory sets an upper bound on what the instruction-tuned model can represent. A feature absent from the base model cannot be reweighted into existence; conversely, a feature present in the base model, whether dormant or latent, becomes a candidate for amplification under fine-tuning. An alignment team concerned about what behaviours an instruction-tuned model might exhibit therefore has a principled reason to audit the base model's SAE features for latent capabilities fine-tuning might surface. This is a specific version of the jailbreak concern [40] that behaviours absent from the IT model's typical outputs may still be recoverable through adversarial prompting, and of the sleeper-agent concern [12] that safety training may mask rather than remove harmful capabilities. The feature-level evidence here supports both in a narrower form: the capabilities fine-tuning operates on are already in the base model, so adversarial prompting that bypasses

fine-tuning should surface base-model features, not instruction-tuning-specific ones.

Safety fine-tuning is a reweighting, not a replacement. The §6.3 finding that IT safety features are substantially drawn from the base-model population implies that safety behaviours are carried by modulations of pre-existing features, not a dedicated circuit added during fine-tuning. This is consistent with the observation [14] that safety fine-tuning can be undone by further fine-tuning on superficially unrelated tasks: if safety is a reweighting rather than a structural addition, any subsequent training that perturbs those weights (in directions that need not be adversarial) can plausibly disrupt it. The feature-level picture does not prove this mechanism, but it is the kind of evidence such a mechanism would imply.

Cross-checkpoint feature transfer is more principled than arbitrary-direction transfer. The §7.3 robustness check found that SAE decoder directions learned on one checkpoint produce structurally distinguishable effects from random directions of matched magnitude when injected into the other checkpoint. This is weak in that it does not validate any specific behavioural steering claim, but non-trivial in suggesting the Gemma Scope 2 feature basis is not arbitrary from the model's perspective: the model processes these directions differently from random perturbations. For practitioners considering using SAE features trained on one checkpoint to steer, probe, or monitor another, this is weak permission to do so, with the caveat that behavioural steering claims require task-specific validation.

7.4 The Pipeline as a Reusable Methodological Contribution

Beyond its empirical findings, the thesis contributes a pipeline applicable to any model pair with matched SAEs. The four stages (activation collection, four-way classification with threshold sensitivity, dual-LLM consensus labelling, targeted differential analysis) are designed for independent reuse. A researcher interested only in classification needs the activation collection and Phase 2 matching code; the behavioural analysis of Phase 4 can be applied to features identified by any means; the dual-LLM labelling is model-agnostic.

Two methodological choices are contributions in their own right. First, the four-way classification using two complementary similarity measures (decoder cosine and activation correlation) is more informative than single-measure classifications. The observation that the modified

class dominates across the entire threshold grid is the classification's main yield: the dominant mode of change between checkpoints is gradual reshaping of an existing feature basis, not replacement or preservation. A single-threshold single-measure classification cannot produce this kind of distributional framing. Second, the dual-LLM labelling treats annotator disagreement as a first-class observable rather than noise. The finding that disagreement concentrates at the `language_syntax` ↔ `instruction_style` boundary turns an apparent weakness of automated labelling into a measurement of how fuzzy the taxonomic boundaries actually are. Single-annotator SAE labelling work would miss this entirely.

Together these choices define a template for matched-SAE comparison studies. Gemma Scope 2 made the Gemma 3 1B comparison possible; a future release providing matched SAEs for a larger model, an RLHF-trained variant, or a non-English model would immediately enable the same pipeline to produce directly comparable results. The main methodological contribution is not the specific Gemma 3 1B finding but the template that made it possible and that can be reused as matched SAEs become more widely available.

7.5 Future Work

Five directions of future work follow from the findings and limitations, in roughly decreasing order of importance.

Behaviourally grounded causal validation. The most important limitation (Chapter 7) is the absence of a causal experiment testing whether candidate features actually drive the behaviours they are selective for. A full version would steer individual features (e.g. cross-model instruction-style features 2006 and 446, or safety-relevant features 262 and 2184) and measure task-specific readouts: refusal rate on JailbreakBench under safety-feature steering, or instruction-following accuracy on Alpaca under instruction-feature steering. The KL-based readout of §7.3 is inadequate because it measures distributional shift without measuring whether the shift is task-meaningful. This experiment should be prioritised above all other future work.

Multi-layer replication. Chapter 6 reports results only at layer 13. Re-running the pipeline at every layer with matched SAEs would produce a layer-by-layer picture of where instruction tuning exerts its effects. The layer profile could distinguish between instruction tuning acting

uniformly across the residual stream, concentrating at middle layers (where semantic features are richest), or concentrating at output layers (where behavioural decisions are made).

Cross-family replication. These findings are a single case study. Replicating on other model families as matched SAEs become available would test whether the wrapper-consistent picture holds beyond Google DeepMind's instruction tuning of Gemma 3 1B. Llama 3, Mistral, and Qwen are plausible targets; the bottleneck is matched-SAE availability, not pipeline complexity.

Cross-checkpoint crosscoders. Replacing the two separate SAEs with a single crosscoder trained jointly on both PT and IT residual streams, as in the sparse-model-diffing literature, would sidestep the matched-SAE comparison problem by design. Crosscoders were not used here because the Gemma Scope 2 crosscoder releases are cross-layer rather than cross-checkpoint, and no PT↔IT crosscoder was publicly available. A future release would allow the classification to be re-derived from a single shared dictionary, and comparing separate-SAE and crosscoder results would be methodologically interesting.

Active-feature threshold sensitivity. A minor unfinished piece: no formal sensitivity grid was computed over the 0.1% active-feature cutoff. Informal checks suggested classification proportions are not strongly sensitive within the 0.05%–0.5% range, but extending the §5.5 grid to include this threshold as a third axis would close the loose end.

7.6 Closing Reflection

This thesis set out to test a contested hypothesis about instruction tuning using a feature-level pipeline, and it reports a descriptive answer consistent with the wrapper reading: at layer 13 of Gemma 3 1B, the features most behaviourally relevant to instruction-following and safety prompts are predominantly reorganised pre-existing features rather than newly introduced ones. The answer is not causal, not multi-layer, not multi-model, and not the last word. It is a concrete, reproducible data point on the feature-level side of a debate that has so far been conducted mostly at the behavioural and circuit level, and a pipeline that can be re-run as matched SAEs become available for other model pairs. The wrapper hypothesis was originally formulated for synthetic tasks and tested with pruning and probing; this thesis extends it to a production-class

instruction-tuned model and tests it with dictionary learning. The convergence of independent methods on compatible readings is, in the author's view, the most significant contribution of this thesis, and the next steps (causal validation, multi-layer replication, cross-family replication) are the natural continuation of the line of work it joins.

A Literature Review Summary

This appendix summarises the studies that most directly motivate the research design and clarifies how each contributes to the present project.

- **Elhage et al. (2022)** [9] establish the superposition framework, explaining why individual neurons are often polysemantic and why neuron-level interpretability is limited.
- **Templeton et al. (2024)** [33] demonstrate that sparse autoencoders can recover large numbers of interpretable, causally meaningful features from frontier language models, making feature-level comparison feasible.
- **Ouyang et al. (2022)** [23] show that post-training techniques such as supervised fine-tuning and RLHF can strongly improve instruction-following, helpfulness, and safety.
- **Zhou et al. (2023)** [38] argue that most capability is learned during pre-training and that a relatively small amount of fine-tuning data can substantially reshape model behaviour, motivating the superficial alignment hypothesis.
- **Du et al. (2025)** [8] provide direct mechanistic evidence that post-training affects some behavioural directions, especially refusal, more than core factual knowledge representations.

B Implications for the Present Study

The literature review motivates the methodology in three ways.

- It justifies the use of sparse autoencoders as the main interpretability tool, because superposition makes individual neurons an unreliable unit of analysis.
- It motivates comparing a base model with its instruction-tuned counterpart, because post-training is the stage where alignment-relevant behavioural changes are introduced.

- It frames the core empirical question of the project: whether post-training creates new interpretable features, suppresses existing ones, or primarily changes the activation patterns of a largely preserved feature set.

References

[1] Anthropic. *Claude Haiku 4.5*. Jan. 1, 2025. URL: https://www.anthropic.com/claude/haiku.

[2] Yuntao Bai et al. *Constitutional AI: Harmlessness from AI Feedback*. Dec. 15, 2022. DOI: 10.48550/arXiv.2212.08073. arXiv: 2212.08073[cs]. URL: http://arxiv.org/abs/2212.08073 (visited on 2026).

[3] Yuntao Bai et al. *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. Apr. 12, 2022. DOI: 10.48550/arXiv.2204.05862. arXiv: 2204.05862[cs]. URL: http://arxiv.org/abs/2204.05862 (visited on 2026).

[4] Trenton Bricken et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. 2023. URL: https://transformer-circuits.pub/2023/monosemantic-features/.

[5] Patrick Chao et al. *JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models*. Mar. 28, 2024. DOI: 10.48550/arXiv.2404.01318. arXiv: 2404.01318[cs.CR]. URL: http://arxiv.org/abs/2404.01318.

[6] Paul Christiano et al. *Deep reinforcement learning from human preferences*. Feb. 17, 2023. DOI: 10.48550/arXiv.1706.03741. arXiv: 1706.03741[stat]. URL: http://arxiv.org/abs/1706.03741 (visited on 2026).

[7] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. Oct. 4, 2023. DOI: 10.48550/arXiv.2309.08600. arXiv: 2309.08600[cs]. URL: http://arxiv.org/abs/2309.08600 (visited on 2026).

[8] Hongzhe Du et al. *How Post-Training Reshapes LLMs: A Mechanistic View on Knowledge, Truthfulness, Refusal, and Confidence*. version: 2. Aug. 11, 2025. DOI: 10.48550/arXiv.2504.02904. arXiv: 2504.02904[cs]. URL: http://arxiv.org/abs/2504.02904 (visited on 2026).

[9] Nelson Elhage et al. *Toy Models of Superposition*. Sept. 21, 2022. DOI: 10.48550/arXiv.2209.10652. arXiv: 2209.10652[cs]. URL: http://arxiv.org/abs/2209.10652 (visited on 2026).

[10] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. June 6, 2024. DOI: 10.48550/arXiv.2406.04093. arXiv: 2406.04093[cs]. URL: http://arxiv.org/abs/2406.04093 (visited on 2026).

[11] Mor Geva et al. "Transformer Feed-Forward Layers Are Key-Value Memories". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2021. Ed. by Marie-Francine Moens et al. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 5484–5495. DOI: 10.18653/v1/2021.emnlp-main.446. URL: https://aclanthology.org/2021.emnlp-main.446/ (visited on 2026).

[12] Evan Hubinger et al. *Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training*. Jan. 17, 2024. DOI: 10.48550/arXiv.2401.05566. arXiv: 2401.05566[cs]. URL: http://arxiv.org/abs/2401.05566 (visited on 2026).

[13] Samyak Jain et al. *Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks*. Aug. 21, 2024. DOI: 10.48550/arXiv.2311.12786. arXiv: 2311.12786[cs]. URL: http://arxiv.org/abs/2311.12786 (visited on 2026).

[14] Samyak Jain et al. *What Makes and Breaks Safety Fine-tuning? A Mechanistic Study*. 2024. arXiv: 2407.10264 [cs.LG]. URL: https://arxiv.org/abs/2407.10264.

[15] Adam Karvonen et al. *SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability*. June 4, 2025. DOI: 10.48550/arXiv.2503.09532. arXiv: 2503.09532[cs]. URL: http://arxiv.org/abs/2503.09532 (visited on 2026).

[16] Simon Kornblith et al. *Similarity of Neural Network Representations Revisited*. July 19, 2019. DOI: 10.48550/arXiv.1905.00414. arXiv: 1905.00414[cs]. URL: http://arxiv.org/abs/1905.00414 (visited on 2026).

[17] Tom Lieberum et al. *Gemma Scope: Open Sparse Autoencoders Everywhere All At Once on Gemma 2*. Aug. 19, 2024. DOI: 10.48550/arXiv.2408.05147. arXiv: 2408.05147[cs]. URL: http://arxiv.org/abs/2408.05147 (visited on 2026).

[18] Stephanie Lin, Jacob Hilton, and Owain Evans. *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. 2022. DOI: 10.48550/arXiv.2109.07958. arXiv: 2109.07958[cs]. URL: http://arxiv.org/abs/2109.07958.

[19] Kevin Meng et al. *Locating and Editing Factual Associations in GPT*. Jan. 13, 2023. DOI: 10.48550/arXiv.2202.05262. arXiv: 2202.05262[cs]. URL: http://arxiv.org/abs/2202.05262 (visited on 2026).

[20] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. Sept. 7, 2013. DOI: 10.48550/arXiv.1301.3781. arXiv: 1301.3781[cs]. URL: http://arxiv.org/abs/1301.3781 (visited on 2026).

[21] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. "Feature Visualization". In: *Distill* 2.11 (Nov. 7, 2017), 10.23915/distill.00007. ISSN: 2476-0757. DOI: 10.23915/distill.00007. URL: https://distill.pub/2017/feature-visualization (visited on 2026).

[22] Chris Olah et al. "Zoom In: An Introduction to Circuits". In: *Distill* 5.3 (Mar. 10, 2020), 10.23915/distill.00024.001. ISSN: 2476-0757. DOI: 10.23915/distill.00024.001. URL: https://distill.pub/2020/circuits/zoom-in (visited on 2026).

[23] Long Ouyang et al. *Training language models to follow instructions with human feedback*. Mar. 4, 2022. DOI: 10.48550/arXiv.2203.02155. arXiv: 2203.02155[cs]. URL: http://arxiv.org/abs/2203.02155 (visited on 2026).

[24] Kiho Park, Yo Joong Choe, and Victor Veitch. *The Linear Representation Hypothesis and the Geometry of Large Language Models*. July 17, 2024. DOI: 10.48550/arXiv.2311.03658. arXiv: 2311.03658[cs]. URL: http://arxiv.org/abs/2311.03658 (visited on 2026).

[25] Guilherme Penedo et al. *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale*. 2024. DOI: 10.48550/arXiv.2406.17557. arXiv: 2406.17557[cs]. URL: http://arxiv.org/abs/2406.17557.

[26] Kenny Peng et al. *Use Sparse Autoencoders to Discover Unknown Concepts, Not to Act on Known Concepts*. June 30, 2025. DOI: 10.48550/arXiv.2506.23845. arXiv: 2506.23845[cs]. URL: http://arxiv.org/abs/2506.23845 (visited on 2026).

[27] Nikhil Prakash et al. *Fine-Tuning Enhances Existing Mechanisms: A Case Study on Entity Tracking*. Feb. 22, 2024. DOI: 10.48550/arXiv.2402.14811. arXiv: 2402.14811[cs]. URL: http://arxiv.org/abs/2402.14811 (visited on 2026).

[28] Alec Radford, Rafal Jozefowicz, and Ilya Sutskever. *Learning to Generate Reviews and Discovering Sentiment*. Apr. 6, 2017. DOI: 10.48550/arXiv.1704.01444. arXiv: 1704.01444[cs]. URL: http://arxiv.org/abs/1704.01444 (visited on 2026).

[29] Rafael Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. July 29, 2024. DOI: 10.48550/arXiv.2305.18290. arXiv: 2305.18290[cs]. URL: http://arxiv.org/abs/2305.18290 (visited on 2026).

[30] Senthooran Rajamanoharan et al. *Improving Dictionary Learning with Gated Sparse Autoencoders*. Apr. 30, 2024. DOI: 10.48550/arXiv.2404.16014. arXiv: 2404.16014[cs]. URL: http://arxiv.org/abs/2404.16014 (visited on 2026).

[31] Adam Scherlis et al. *Polysemanticity and Capacity in Neural Networks*. Mar. 25, 2025. DOI: 10.48550/arXiv.2210.01892. arXiv: 2210.01892[cs]. URL: http://arxiv.org/abs/2210.01892 (visited on 2026).

[32] Rohan Taori et al. *Stanford Alpaca*. Instruction-following dataset repository. 2023. URL: https://github.com/tatsu-lab/stanford_alpaca.

[33] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. 2024. URL: https://transformer-circuits.pub/2024/scaling-monosemanticity/.

[34] Kevin Wang et al. *Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small*. Nov. 1, 2022. DOI: 10.48550/arXiv.2211.00593. arXiv: 2211.00593[cs]. URL: http://arxiv.org/abs/2211.00593 (visited on 2026).

[35] Jason Wei et al. *Finetuned Language Models Are Zero-Shot Learners*. Feb. 8, 2022. DOI: 10.48550/arXiv.2109.01652. arXiv: 2109.01652[cs]. URL: http://arxiv.org/abs/2109.01652 (visited on 2026).

[36] B. L. Welch. "The Generalization of "Student's" Problem When Several Different Population Variances Are Involved". In: *Biometrika* 34.1-2 (Jan. 1, 1947), pp. 28–35. DOI: 10.1093/biomet/34.1-2.28. URL: https://doi.org/10.1093/biomet/34.1-2.28.

[37] Yang Xu et al. *Tracking the Feature Dynamics in LLM Training: A Mechanistic Study*. 2025. arXiv: 2412.17626 [cs.LG]. URL: https://arxiv.org/abs/2412.17626 (visited on 2026).

[38] Chunting Zhou et al. *LIMA: Less Is More for Alignment*. May 18, 2023. DOI: 10.48550/arXiv.2305.11206. arXiv: 2305.11206[cs]. URL: http://arxiv.org/abs/2305.11206 (visited on 2026).

[39] Daniel M. Ziegler et al. *Fine-Tuning Language Models from Human Preferences*. Jan. 8, 2020. DOI: 10.48550/arXiv.1909.08593. arXiv: 1909.08593[cs]. URL: http://arxiv.org/abs/1909.08593 (visited on 2026).

[40] Andy Zou et al. *Universal and Transferable Adversarial Attacks on Aligned Language Models*. Dec. 20, 2023. DOI: 10.48550/arXiv.2307.15043. arXiv: 2307.15043[cs]. URL: http://arxiv.org/abs/2307.15043 (visited on 2026).

[41] Andy Zou et al. *Representation Engineering: A Top-Down Approach to AI Transparency*. Mar. 3, 2025. DOI: 10.48550/arXiv.2310.01405. arXiv: 2310.01405[cs]. URL: http://arxiv.org/abs/2310.01405 (visited on 2026).